VOLUME 02: TRACK 1 - HOST ENGINE & VIRTUAL MACHINE MANUAL

Chapter 8: Track 1 - Bytecode Assembler & Lowering

Instruction Encodings, Lowering Lexicon Tapes to Machine Words, Baton Passes

> ***Witness:*** *Colossians 2:19 (KJV) - The whole body fitly joined together*

SYSTEM TRACK
<TRACK 1> : {Host Engine & Virtual Machine}

CHAPTER IDENTIFIER
CH 08

NOTEBOOKLM WORD COUNT
17,369 words

CATALOGED CHAPTER FILES
17 files (4,315 lines)

RECOMMENDED NOTEBOOKLM COGNITIVE QUERY

"Trace the lowering pipeline from high-level process tape rolls to 64-bit machine instruction words."

CHAPTER MANIFEST & INDEX (17 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/README.md**	MD	136	1,065	8.6
2	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/core.rs**	RS	529	2,197	26.2
3	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/mod.rs**	RS	94	474	4.3
4	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/lowering/mod.rs**	RS	46	276	2.4
5	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/lowering/omni.rs**	RS	227	834	10.2
6	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/lowering/roll.rs**	RS	387	1,403	17.3
7	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/lowering/tape.rs**	RS	91	378	3.8
8	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/universal/isa.rs**	RS	357	1,693	15.5
9	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/universal/mod.rs**	RS	52	194	2.0
10	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/universal/types.rs**	RS	210	900	8.2
11	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/transpiler/baremetal.rs**	RS	1,113	3,548	32.1
12	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/transpiler/c.rs**	RS	297	1,211	15.5
13	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/transpiler/json.rs**	RS	109	475	5.0
14	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/transpiler/listing.rs**	RS	129	537	5.9
15	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/transpiler/lua.rs**	RS	297	1,189	15.8
16	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/transpiler/mod.rs**	RS	59	217	2.1
17	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/transpiler/omni.rs**	RS	182	778	9.2

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/README.md | MARKDOWN / DOC | 136 LINES | 1065 WORDS | 8.6 KB

OmniCode Runtime Physical Assembler Subsystem

```
================================================================================
BLOCK 1: METADATA & ARCHITECTURAL MANIFESTO
================================================================================
Subsystem: OmniCode Host Engine & Virtual Machine Physical Assembler
Layer:     Silicon Lowering, Memory Allocation & Multi-Target Transpiler Engine
Path:      <TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/
================================================================================
```

> ***"And he gave unto Moses, when he had made an end of communing with him upon mount Sinai, two tables of testimony, tables of stone, written with the finger of God."***
> *— Exodus 31:18 (KJV)*
> *(The immutable inscription of atomic runes onto bounded physical tablets).*
>
> ***"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house."***
> *— Proverbs 24:27 (KJV)*
> *(The discipline of proving atomic sub-engines and verification invariants before constructing complex systems).*
>
> ***"Remove not the ancient landmark, which thy fathers have set."***
> *— Proverbs 22:28 (KJV)*
> *(The preservation of historical memory boundaries, silicon addresses, and machine constants).*

1. Architectural Mission

The **Physical Assembler Subsystem** is the transduction bridge between abstract ontological expressions (`+`) and concrete silicon hardware execution (`-`). It transforms high-level procedural process ribbons (`.tape`) and declarative spatial architecture reels (`.omni`) into discrete, 64-bit balanced-ternary machine words ([`Word64`]), allocates physical storehouse memory partitions, binds linguistic pronouns to CPU hardware registers, and emits multi-target runtime artifacts (bare-metal BIOS MBR bootloaders, Multiboot ELF kernels, JSON ASTs, and assembly listings).

By decomposing the monolithic 1,930-line assembler into four specialized architectural tiers:

1. **Universal Substrate (`universal/`)**: Houses the hardware instruction set architecture ([`MachineOp`]), 13 named hardware registers ([`MachineRegister`]), balanced-ternary machine word packaging, and core data containers ([`PhysicalSymbol`], [`AssembledInstruction`], [`AssembledTape`]).
2. **Lowering Pipelines (`lowering/`)**: Compiles atomic procedural statements (`roll.rs`), entire process ribbons (`tape.rs`), and 4-Block spatial reels (`omni.rs`) into physical silicon instructions with memory bounds tracking.
3. **Multi-Target Transpilers (`transpiler/`)**: Projects lowered tapes into annotated assembly listings (`listing.rs`), IDE hovercard JSON (`json.rs`), bidirectional `.tape` <-> `.omni` translations (`omni.rs`), and bare-metal x86 Real Mode / Multiboot assembly (`baremetal.rs`).
4. **Core Coordinator (`core.rs`)**: Orchestrates the [`OmniAssembler`], handles filesystem path discovery, and houses the comprehensive regression test suite.

2. Topological Layer Structure

```
src/runtime/assembler/
├── universal/                <-- Tier 1: Hardware ISA & Fundamental Data Containers
│   ├── isa.rs                <-- MachineOp, MachineRegister, Pronoun/Polarity Maps, Word64 Synthesis
│   ├── types.rs              <-- PhysicalSymbol, AssembledInstruction, AssembledTape, Hardware Bounds
│   └── mod.rs                <-- Re-exports Universal ISA & Types
│
├── lowering/                 <-- Tier 2: Tape & Omni Statement Lowering Pipelines
```

```
│   ├── roll.rs                <-- Atomic Single-Roll Lowering (Memory Offsets, Delimiters, SVO Verbs)
│   ├── tape.rs                <-- Full Procedural .tape Ribbon Lowering Loop
│   ├── omni.rs                <-- Declarative 4-Block .omni Spatial Reel Parser & Lowerer
│   └── mod.rs                 <-- Re-exports Lowering Pipelines
│
├── transpiler/                <-- Tier 3: Multi-Target Code & Serializer Generation
│   ├── listing.rs             <-- Human-Readable Annotated Assembly Listings & Silicon Words
│   ├── json.rs                <-- Structured JSON ASTs for Language Server & Debuggers
│   ├── omni.rs                <-- Bidirectional Decompilation (.tape <--> .omni)
│   ├── baremetal.rs           <-- 16-Bit Real Mode BIOS MBR Bootloader & 32-Bit Multiboot ELF Assembly
│   └── mod.rs                 <-- Re-exports Multi-Target Transpilers
│
├── core.rs                    <-- Tier 4: Main OmniAssembler Struct, assemble_file & Verification Tests
├── mod.rs                     <-- Subsystem Public Facade (100% Backward Compatible)
└── README.md                  <-- Architectural Manifesto & Specification
```

3. Subsystem Breakdown

3.1 Universal (`universal/`)

- **Hardware Opcodes (`MachineOp`)**: Defines discrete silicon operations (`Nop` , `RefAbstract` , `AllocConcrete` , `TransduceVector` , `FeedbackVector` , `PartitionSlice` , `SynthesizeLayer` , `BindChannel` , `CastBoundary` , `IntrospectSelf` , `RecurseItself` , `AssertCapability`).
- **Named Silicon Registers (`MachineRegister`)**: 13 physical CPU registers with bidirectional parsing and backward compatibility aliases:
- `HEAD` (0 / `%head` / `%ip` / `HERE`)
- `HOME` (1 / `%home` / `%fp` / `ORIGIN`)
- `SPOOL` (2 / `%spool` / `%sp` / `OWNER`)
- `IT` (3 / `%it` / `%acc`)
- `SELF` (4 / `%self` / `sl`)
- `CALLER` (5 / `%caller` / `cl`)
- `OTHER` (6 / `%other` / `%swarm`)
- `STATE` (7 / `%state` / `%flags`)
- `TURN` (8 / `%turn` / `step`)
- `BOUND` (9 / `%bound` / `limit`)
- `FROM` (10 / `%from` / `input`)
- `INTO` (11 / `%into` / `output`)
- `DISCARD` (12 / `%discard` / `void`)
- **Word64 Synthesis**: Packs 32 trits payload, QuadState status, Polarity, Boundary rank, and a 7-bit checksum into a single 64-bit machine word.

3.2 Lowering Pipelines (`lowering/`)

- **`roll.rs`** : Validates delimiter boundary nesting (`[` , `{` , `<`) via [`OmniDebugger`], assigns concrete RAM partition offsets (`@ 0x0000`), binds agentic pronouns, and maps SVO action verbs (`INSCRIBE` , `TRANSDUCE` , `PROJECT` , `SEQUENCE`) to hardware opcodes.
- **`tape.rs`** : Iterates through procedural rolls, carving memory partitions according to Tape Head covenants and validating that final execution commits in `+1:PROCEED` .
- **`omni.rs`** : Parses multi-block spatial `.omni` files, extracts metadata and block structure, and lowers declarations into executable silicon tapes.

3.3 Multi-Target Transpilers (`transpiler/`)

- **Listing & Dumps (`listing.rs`)**: Emits formatted assembly tables and silicon hex word streams.
- **Tooling ASTs (`json.rs`)**: Emits structured JSON payloads containing symbol tables and instruction metadata for language servers and IDE hovercards.
- **Format Transduction (`omni.rs`)**: Converts procedural `.tape` files into declarative 4-Block `.omni` reels and vice versa.
- **Bare-Metal OS (`baremetal.rs`)**: Generates 16-bit Real Mode MBR boot sectors and 32-bit Multiboot ELF source code targeting MilleniumOS on physical x86 or QEMU hypervisors.

3.4 Core Coordinator (`core.rs`)

- **`OmniAssembler`** : Encapsulates [`DiagnosticLogger`], [`OmniDebugger`], and candidate search paths (`lexicon/` , `lib/`).

- **`assemble_file`**: Universal filesystem entry point that detects `.tape` or `.omni` extensions and directs execution to the appropriate lowering pipeline.
- **Verification Tests**: Houses 9 regression oracles proving the integrity of the entire modular system.

4. The Physical Baton Pass Contract

```
graph TD
    A["Source File (.tape / .omni)"] --> B["OmniAssembler::assemble_file"]
    B --> C{"File Extension"}
    C -->|tape| D["assemble_tape"]
    C -->|omni| E["assemble_omni"]
    D --> F["assemble_roll (per statement)"]
    E --> F
    F --> G["OmniDebugger (Delimiter Stack Invariant)"]
    G --> H["Symbol & Memory Partition Allocator (@ offset)"]
    H --> I["Word64 Balanced-Ternary Packaging"]
    I --> J["AssembledTape (+1:PROCEED Covenant Seal)"]
    J --> K["Downstream VM / QEMU / Transpiler Emission"]
```

5. Architectural Invariants

1. **Deterministic Memory Offsets**: Every concrete entity (`-`) receives a deterministic cell index within its partition, beginning at `@ 0x0000`.
2. **Boundary Closure**: Unmatched or leaking brackets (`[` , `{` , `<`) trigger immediate failpoints before any machine code is emitted.
3. **Covenant Affirmation**: Execution tapes must commit in `+1:PROCEED` ; unsealed or ambiguous tapes are rejected.
4. **Universal Transduction**: All three triad representations (`.tape` , `.omni` , `Word64`) remain strictly isomorphic and bidirectionally convertible without information loss.

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/core.rs RS 529 LINES 2197 WORDS 26.2 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Physical Assembler Core Coordinator, Lifecycle & Hardware Invariants )
// ============================================================================
//! OmniCode Runtime: Physical Assembler Core Coordinator
//!
//! # Architectural Scope & Purpose:
//! Encapsulates the main [`OmniAssembler`] coordinator struct:
//! 1. Telemetry and failpoint recording via [`DiagnosticLogger`].
//! 2. Structural boundary invariant enforcement via [`OmniDebugger`].
//! 3. Probing candidate lexicon search paths (`lexicon/`, `lib/`).
//! 4. High-level file-based assembly entry points (`assemble_file`).
//! 5. Transpilation convenience wrappers (`transpile_omni_source_to_tape`, `transpile_omni_source_to_asm`).
//! 6. Comprehensive unit test oracles covering all 13 hardware registers, SVO lowering, and 28 lexicon tapes.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of physical machine code onto bounded hardware substrates).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The inviolable preservation of historical silicon addresses, MMIO ports, and opcodes).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Silicon Truth**: Coordinates all memory allocation, register binding, and machine word packaging.
//! 2. **Baton Pass**: Connects callers (CLI, REPL, JIT) to lowered silicon layouts and bare-metal emitters.
// ============================================================================

use std::path::{Path, PathBuf};

use crate::core::tape::Tape;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::DiagnosticLogger;
use crate::runtime::assembler::transpiler::baremetal::transpile_tape_to_asm;
use crate::runtime::assembler::transpiler::omni::transpile_omni_to_tape;
use crate::runtime::assembler::universal::types::AssembledTape;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( OmniAssembler Coordinator Struct & Constructor )
// ============================================================================

/// Encapsulates the runtime diagnostic logger, boundary invariant debugger ([`OmniDebugger`]),
/// and candidate lexicon search paths required to resolve and lower ontological symbols into silicon memory.
#[derive(Debug, Clone)]
pub struct OmniAssembler {
    /// Attached telemetry and error interception logger.
    pub logger: DiagnosticLogger,
    /// Delimiter boundary stack tracking engine for structural invariants (`[...]`, `<...>`).
    pub debugger: OmniDebugger,
    /// Candidate filesystem search directories for modular lexicon resolution.
    pub lexicon_paths: Vec<PathBuf>,
}

impl OmniAssembler {
    /// Constructs a new physical silicon assembler bound to the given diagnostic logger and debugger.
    ///
    /// Automatically probes standard candidate lexicon and library paths (`lexicon/`, `lib/`)
    /// to support cross-track ontological symbol resolution.
    pub fn new(logger: DiagnosticLogger, debugger: OmniDebugger) -> Self {
        let mut lexicon_paths = Vec::new();
        let candidate_dirs = ["lexicon", "lib"];
        for dir in &candidate_dirs {
            let p = PathBuf::from(dir);
            if p.exists() {
                lexicon_paths.push(p);
            }
        }
        Self {
```

```
            logger,
            debugger,
            lexicon_paths,
        }
    }

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( High-Level Assembly & Transpilation Execution Pipelines )
// ============================================================================

    /// Transpiles a raw declarative .omni string directly into an executable .tape string.
    pub fn transpile_omni_source_to_tape(&mut self, source: &str) -> Result<String, String> {
        let assembled = self.assemble_omni(source)?;
        Ok(transpile_omni_to_tape(&assembled))
    }

    /// Transpiles a raw declarative .omni string directly into bare-metal x86 bootstrap assembly (`.asm`).
    pub fn transpile_omni_source_to_asm(&mut self, source: &str) -> Result<String, String> {
        let assembled = self.assemble_omni(source)?;
        Ok(transpile_tape_to_asm(&assembled))
    }

    /// Assembles a .tape or .omni file directly from disk.
    pub fn assemble_file(&mut self, path: impl AsRef<Path>) -> Result<AssembledTape, String> {
        let p = path.as_ref();
        let ext = p.extension().and_then(|e| e.to_str()).unwrap_or("");
        match ext {
            "tape" => {
                let tape = Tape::from_file(p, self.logger.clone())?;
                self.assemble_tape(&tape)
            }
            "omni" => {
                let source = std::fs::read_to_string(p)
                    .map_err(|e| format!("Failed to read .omni file '{}': {e}", p.display()))?;
                self.assemble_omni(&source)
            }
            other => Err(format!(
                "Unsupported file extension '.{}' for '{}' (expected .tape or .omni)",
                other, p.display()
            )),
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Baton Pass Contract, Invariant Verification Proofs & Unit Oracles )
// ============================================================================

// ----------------------------------------------------------------------------
// 4.1. The Physical Baton Pass Contract
// ----------------------------------------------------------------------------
// BATON PASS CONTRACT:
// 1. INPUT INGESTION: `OmniAssembler::assemble_file` or `assemble_tape` ingests high-level
//    procedural tapes (`.tape`) or spatial reels (`.omni`) from callers (CLI, REPL, JIT).
// 2. ATOMIC LOWERING: `assemble_roll` tokenizes each statement, asserts boundary integrity via
//    `OmniDebugger`, resolves polarities (`+` abstract vs `-` concrete), assigns physical memory
//    partition offsets (`@ 0x0000`), binds agentic pronouns to hardware CPU registers (`%self`,
//    `%caller`, `%acc`, `%swarm`), and packs 64-bit balanced-ternary machine words (`Word64`).
// 3. INVARIANT & SEAL CERTIFICATION: The tape head covenant and commit status are evaluated,
//    guaranteeing that execution only seals in affirmative `+1:PROCEED`.
// 4. OUTPUT ARTIFACT SYNTHESIS: `AssembledTape` provides universal emission methods (`format_listing`,
//    `format_words`, `format_json`, `format_omni`, `to_machine_words`).
// 5. BATON HANDOFF: The resulting `AssembledTape` or `Vec<Word64>` is handed off to downstream
//    virtual machine runners, the QEMU bare-metal hypervisor, or CLI output serializers.

#[cfg(test)]
mod tests {
    use super::*;
    use std::collections::HashMap;
    use crate::core::tape::Roll;
    use crate::core::ternary::QuadState;
    use crate::runtime::assembler::universal::isa::{MachineOp, MachineRegister};

    /// Test 1: Verifies that the physical assembler lowers living world ontological tapes,
    /// synthesizing concrete memory cell offsets and affirmative PROCEED commit status.
    #[test]
```

```
    fn test_assembler_lowers_living_world_tape() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let mut asm = OmniAssembler::new(logger, debugger);

        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/living_world.tape");
        if path.exists() {
            let assembled = asm.assemble_file(&path).expect("Assembling living_world.tape");
            assert_eq!(assembled.head.name, "LivingWorldInstantiation");
            assert!(assembled.symbols.contains_key("Dog"));
            assert_eq!(assembled.commit_status, QuadState::Proceed);
            let listing = assembled.format_listing();
            assert!(listing.contains("ASSEMBLED MACHINE TAPE"));
            assert!(listing.contains("Dog"));
        }
    }

    /// Test 2: Verifies polarity transformation Lowering (`+Archetype --> Concrete-`).
    /// Ensures that abstract entities register in the symbol table with Abstract polarity,
    /// and concrete entities receive designated silicon memory cells.
    #[test]
    fn test_assembler_lowers_polarity_tape() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let mut asm = OmniAssembler::new(logger, debugger);

        let path = crate::core::filetype::resolve_track_path("lexicon/operators/polarity.tape");
        if path.exists() {
            let assembled = asm.assemble_file(&path).expect("Assembling polarity.tape");
            assert_eq!(assembled.head.name, "PolarityTransformation");
            assert!(assembled.symbols.contains_key("Archetype"));
            assert!(assembled.symbols.contains_key("Concrete"));
            assert_eq!(assembled.commit_status, QuadState::Proceed);
        }
    }

    /// Test 3: Exhaustive regression oracle across the entire modular Track 2 Lexicon.
    /// Assembles all 30 foundational grammar and kernel tapes, asserting that every single
    /// tape seals with affirmative `+1:PROCEED` without unhandled delimiter or syntax errors.
    #[test]
    fn test_assembler_lowers_all_modular_lexicon_tapes() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let mut asm = OmniAssembler::new(logger, debugger);

        let tape_paths = [
            "lexicon/operators/polarity.tape",
            "lexicon/operators/casting.tape",
            "lexicon/01_nouns/atomic_units.tape",
            "lexicon/01_nouns/vessels.tape",
            "lexicon/01_nouns/agents.tape",
            "lexicon/01_nouns/living_world.tape",
            "lexicon/02_pronouns/self_handles.tape",
            "lexicon/02_pronouns/wildcards.tape",
            "lexicon/03_verbs/axial_vectors.tape",
            "lexicon/03_verbs/scale_vectors.tape",
            "lexicon/03_verbs/diagonal_vectors.tape",
            "lexicon/03_verbs/relational_being.tape",
            "lexicon/03_verbs/arithmetic_verbs.tape",
            "lexicon/04_adjectives/quad_states.tape",
            "lexicon/04_adjectives/invariant_axioms.tape",
            "lexicon/04_adjectives/proportions.tape",
            "lexicon/05_adverbs/negation.tape",
            "lexicon/05_adverbs/cadence_policies.tape",
            "lexicon/06_prepositions/spatial_sites.tape",
            "lexicon/06_prepositions/ranges.tape",
            "lexicon/07_conjunctions/inseparable_pairs.tape",
            "lexicon/07_conjunctions/pipeline_junctions.tape",
            "lexicon/07_conjunctions/caveat_branches.tape",
            "lexicon/07_conjunctions/alternative_branches.tape",
            "lexicon/07_conjunctions/relational_attributions.tape",
            "lexicon/08_interjections/kernel_traps.tape",
            "lexicon/08_interjections/teleology_asides.tape",
            "lexicon/03_verbs/cinematic_track_loop.tape",
            "lexicon/operators/boot_kernel.tape",
            "lib/kernel/boot.tape",
        ];

```

```
        for path_str in &tape_paths {
            let path = crate::core::filetype::resolve_track_path(path_str);
            assert!(path.exists(), "Tape file does not exist: {}", path_str);
            let assembled = asm.assemble_file(&path).unwrap_or_else(|e| panic!("Failed to assemble {}: {}", path_str, e));
            assert_eq!(assembled.commit_status, QuadState::Proceed, "Tape {} did not commit in PROCEED", path_str);
            assert!(!assembled.instructions.is_empty(), "Tape {} had no instructions", path_str);
        }
    }

    /// Test 4: Verifies grammatical pronoun lowering into physical hardware CPU registers.
    /// Asserts that self-handles (`self`) map to `%self` (Register 4), wildcards and
    /// antecedents (`it`) map to `%acc` (Register 3), and callers map to `%caller`.
    #[test]
    fn test_assembler_lowers_pronouns_and_registers() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let mut asm = OmniAssembler::new(logger, debugger);

        let path = crate::core::filetype::resolve_track_path("lexicon/02_pronouns/self_handles.tape");
        if path.exists() {
            let assembled = asm.assemble_file(&path).expect("Assembling self_handles.tape");
            let self_roll = assembled.instructions.iter().find(|i| i.original_text.contains("self"));
            assert!(self_roll.is_some(), "Expected self roll in instructions");
            assert_eq!(self_roll.unwrap().register, Some(MachineRegister::SELF));
        }

        let wild_path = crate::core::filetype::resolve_track_path("lexicon/02_pronouns/wildcards.tape");
        if wild_path.exists() {
            let assembled = asm.assemble_file(&wild_path).expect("Assembling wildcards.tape");
            let it_roll = assembled.instructions.iter().find(|i| i.original_text.contains("it"));
            assert!(it_roll.is_some(), "Expected it roll in instructions");
            assert_eq!(it_roll.unwrap().register, Some(MachineRegister::ACC));
        }
    }

    /// Test 5: Verifies universal serialization methods on `AssembledTape`.
    /// Validates `format_words()` (physical silicon hex dump), `format_json()` (IDE/tooling AST),
    /// and `format_omni()` (decompilation/transpilation into declarative 4-Block spatial reels).
    #[test]
    fn test_assembler_universal_formatting_methods() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let mut asm = OmniAssembler::new(logger, debugger);

        let path = crate::core::filetype::resolve_track_path("lib/kernel/boot.tape");
        if path.exists() {
            let assembled = asm.assemble_file(&path).expect("Assembling boot.tape");

            // 1. Format Words
            let words_dump = assembled.format_words();
            assert!(words_dump.contains("PHYSICAL SILICON WORD STREAM"));
            assert!(words_dump.contains("TOTAL WORDS:"));

            // 2. Format JSON
            let json_dump = assembled.format_json();
            assert!(json_dump.contains("\"tape\":"));
            assert!(json_dump.contains("\"symbols\":"));
            assert!(json_dump.contains("\"instructions\":"));

            // 3. Format Omni (Transpilation)
            let omni_dump = assembled.format_omni();
            assert!(omni_dump.contains("BLOCK 1: METADATA BLOCK"));
            assert!(omni_dump.contains("BLOCK 2: OPENING BLOCK"));
            assert!(omni_dump.contains("BLOCK 3: BODY BLOCK"));
            assert!(omni_dump.contains("BLOCK 4: CLOSING BLOCK"));
            assert!(omni_dump.contains("commit_status [ PROCEED ]"));
        }
    }

    /// Test 6: Verifies declarative `.omni` spatial reel lowering into physical machine layout.
    /// Confirms that `assemble_omni` correctly parses multi-block `.omni` code, constructs
    /// the symbol table and instruction stream, and verifies the Closing block seal.
    #[test]
    fn test_assembler_lowers_declarative_omni_reel() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let mut asm = OmniAssembler::new(logger, debugger);
```

```
308
309         let path = crate::core::filetype::resolve_track_path("lexicon/03_verbs/axial_vectors.omni");
310         if path.exists() {
311             let assembled = asm.assemble_file(&path).expect("Assembling axial_vectors.omni");
312             assert_eq!(assembled.commit_status, QuadState::Proceed);
313             assert!(!assembled.instructions.is_empty(), "Must produce instruction stream");
314             assert_eq!(assembled.head.who, "OmniAssembler");
315         }
316     }
317
318     /// Test 7: Verifies standalone atomic single-roll lowering via `assemble_roll`.
319     /// Confirms that an individual roll can be lowered outside of a Tape container,
320     /// correctly allocating memory partition offsets and updating the physical symbol table.
321     #[test]
322     fn test_assembler_assemble_roll_standalone() {
323         let logger = DiagnosticLogger::new(false);
324         let debugger = OmniDebugger::new(logger.clone(), 100);
325         let mut asm = OmniAssembler::new(logger, debugger);
326
327         let roll = Roll {
328             index: 1,
329             line_number: 10,
330             text: "roll_1 [ +Memory --> Memory- @ 0 ]".to_string(),
331             status: QuadState::Proceed,
332             is_head: false,
333         };
334
335         let mut symbols = HashMap::new();
336         let mut next_offset = 0;
337         let mut commit_status = QuadState::Await;
338
339         let inst = asm.assemble_roll(
340             &roll,
341             0,
342             "TestPartition",
343             &mut symbols,
344             &mut next_offset,
345             &mut commit_status,
346         ).expect("Assembling standalone roll");
347
348         assert_eq!(inst.op, MachineOp::AllocConcrete);
349         assert!(symbols.contains_key("Memory"));
350         assert_eq!(symbols["Memory"].offset, Some(0));
351         assert_ne!(inst.word.raw, 0xFFFFFFFF_FFFFFFFF);
352     }
353
354     /// Test 8: Verifies full coverage of all 13 Named Silicon Registers,
355     /// backward-compatible aliases, ALL CAPS and lowercase duality, affix handling,
356     /// and dynamic roll register binding.
357     #[test]
358     fn test_named_silicon_registers_coverage() {
359         let registers = [
360             (MachineRegister::HEAD, 0, "HEAD", "%head"),
361             (MachineRegister::HOME, 1, "HOME", "%home"),
362             (MachineRegister::SPOOL, 2, "SPOOL", "%spool"),
363             (MachineRegister::IT, 3, "IT", "%it"),
364             (MachineRegister::SELF, 4, "SELF", "%self"),
365             (MachineRegister::CALLER, 5, "CALLER", "%caller"),
366             (MachineRegister::OTHER, 6, "OTHER", "%other"),
367             (MachineRegister::STATE, 7, "STATE", "%state"),
368             (MachineRegister::TURN, 8, "TURN", "%turn"),
369             (MachineRegister::BOUND, 9, "BOUND", "%bound"),
370             (MachineRegister::FROM, 10, "FROM", "%from"),
371             (MachineRegister::INTO, 11, "INTO", "%into"),
372             (MachineRegister::DISCARD, 12, "DISCARD", "%discard"),
373         ];
374
375         for (reg, expected_idx, expected_hw, expected_name) in registers {
376             assert_eq!(reg.index(), expected_idx);
377             assert_eq!(reg.hardware_name(), expected_hw);
378             assert_eq!(reg.name(), expected_name);
379             assert_eq!(MachineRegister::from_name(expected_hw), Some(reg));
380             assert_eq!(MachineRegister::from_name(&expected_hw.to_lowercase()), Some(reg));
381             assert_eq!(MachineRegister::from_name(&format!("%{}", expected_hw.to_lowercase())), Some(reg));
382             assert_eq!(MachineRegister::from_name(&format!("+{}", expected_hw)), Some(reg));
383             assert_eq!(MachineRegister::from_name(&format!("-{}", expected_hw)), Some(reg));
384         }
385
```

```
        // Backward compatibility constants
        assert_eq!(MachineRegister::IP, MachineRegister::HEAD);
        assert_eq!(MachineRegister::FP, MachineRegister::HOME);
        assert_eq!(MachineRegister::SP, MachineRegister::SPOOL);
        assert_eq!(MachineRegister::ACC, MachineRegister::IT);
        assert_eq!(MachineRegister::FLAGS, MachineRegister::STATE);
        assert_eq!(MachineRegister::HERE, MachineRegister::HEAD);
        assert_eq!(MachineRegister::ORIGIN, MachineRegister::HOME);
        assert_eq!(MachineRegister::OWNER, MachineRegister::SPOOL);
        assert_eq!(MachineRegister::SWARM, MachineRegister::OTHER);
        assert_eq!(MachineRegister::STEP, MachineRegister::TURN);
        assert_eq!(MachineRegister::INPUT, MachineRegister::FROM);
        assert_eq!(MachineRegister::OUTPUT, MachineRegister::INTO);

        // Natural, positional, and shorthand aliases
        assert_eq!(MachineRegister::from_name("ip"), Some(MachineRegister::HEAD));
        assert_eq!(MachineRegister::from_name("here"), Some(MachineRegister::HEAD));
        assert_eq!(MachineRegister::from_name("origin"), Some(MachineRegister::HOME));
        assert_eq!(MachineRegister::from_name("spool"), Some(MachineRegister::SPOOL));
        assert_eq!(MachineRegister::from_name("owner"), Some(MachineRegister::SPOOL));
        assert_eq!(MachineRegister::from_name("turn"), Some(MachineRegister::TURN));
        assert_eq!(MachineRegister::from_name("step"), Some(MachineRegister::TURN));
        assert_eq!(MachineRegister::from_name("bound"), Some(MachineRegister::BOUND));
        assert_eq!(MachineRegister::from_name("from"), Some(MachineRegister::FROM));
        assert_eq!(MachineRegister::from_name("input"), Some(MachineRegister::FROM));
        assert_eq!(MachineRegister::from_name("into"), Some(MachineRegister::INTO));
        assert_eq!(MachineRegister::from_name("output"), Some(MachineRegister::INTO));
        assert_eq!(MachineRegister::from_name("_"), Some(MachineRegister::DISCARD));
        assert_eq!(MachineRegister::from_name("void"), Some(MachineRegister::DISCARD));

        // Verify that assemble_roll binds Named Silicon Registers directly
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let mut asm = OmniAssembler::new(logger, debugger);

        let roll_turn = Roll {
            index: 1,
            line_number: 1,
            text: "roll_turn [ TURN plus 1 --> TURN ]".to_string(),
            status: QuadState::Proceed,
            is_head: false,
        };
        let mut syms = HashMap::new();
        let mut off = 0;
        let mut commit = QuadState::Await;
        let inst_turn = asm.assemble_roll(&roll_turn, 0, "Part", &mut syms, &mut off, &mut commit).expect("Assemble
TURN");
        assert_eq!(inst_turn.register, Some(MachineRegister::TURN));

        let roll_bound = Roll {
            index: 2,
            line_number: 2,
            text: "roll_bound [ BOUND ]".to_string(),
            status: QuadState::Proceed,
            is_head: false,
        };
        let inst_bound = asm.assemble_roll(&roll_bound, 1, "Part", &mut syms, &mut off, &mut commit).expect("Assemble
BOUND");
        assert_eq!(inst_bound.register, Some(MachineRegister::BOUND));
    }

    /// Test 9: Verifies SVO action verbs (INSCRIBE, TRANSDUCE, PROJECT, SEQUENCE),
    /// bijective Radix-3 equivalence (<=>), and MMIO port lowering for Character tapes.
    #[test]
    fn test_character_tape_svo_lowering_and_equivalence() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let mut asm = OmniAssembler::new(logger, debugger);
        let mut syms = HashMap::new();
        let mut off = 0;
        let mut commit = QuadState::Await;

        // 1. ROLL 1: INSCRIBE verb -> AllocConcrete / INSCRIBE_CELL
        let roll1 = Roll {
            index: 1,
            line_number: 1,
            text: "[ ROLL 1 ] : [ Inscribe ] : { Glyph ] --> INSCRIBE [ Cell ] = [ Character` ]".to_string(),
            status: QuadState::Proceed,
```

```
                is_head: false,
            };
            let inst1 = asm.assemble_roll(&roll1, 0, "CharacterTape", &mut syms, &mut off, &mut commit).expect("Assemble
ROLL 1");
            assert_eq!(inst1.op_code, "INSCRIBE_CELL");
            assert_eq!(inst1.op, MachineOp::AllocConcrete);

            // 2. ROLL 2: Bijective equivalence <=> and TRANSDUCE verb -> TransduceVector / TRANSDUCE_SUBSTRATE
            let roll2 = Roll {
                index: 2,
                line_number: 2,
                text: "[ ROLL 2 ] : [ Transduce ] : [ Transducer ]: [ Character` { 8 Bits } ] <=> TRANSDUCE [ Cell` { 5
Trits } ]".to_string(),
                status: QuadState::Proceed,
                is_head: false,
            };
            let inst2 = asm.assemble_roll(&roll2, 1, "CharacterTape", &mut syms, &mut off, &mut commit).expect("Assemble
ROLL 2");
            assert_eq!(inst2.op_code, "TRANSDUCE_SUBSTRATE");
            assert_eq!(inst2.op, MachineOp::TransduceVector);

            // 3. ROLL 4: PROJECT verb and Target -> TransduceVector / PROJECT_TARGET with register INTO
            let roll4 = Roll {
                index: 4,
                line_number: 4,
                text: "[ ROLL 4 ] : [ Project ] : [ Character` } --> PROJECT [ Target ] = [ Target` }".to_string(),
                status: QuadState::Proceed,
                is_head: false,
            };
            let inst4 = asm.assemble_roll(&roll4, 3, "CharacterTape", &mut syms, &mut off, &mut commit).expect("Assemble
ROLL 4");
            assert_eq!(inst4.op_code, "PROJECT_TARGET");
            assert_eq!(inst4.op, MachineOp::TransduceVector);
            assert_eq!(inst4.register, Some(MachineRegister::INTO));

            // 4. ROLL 5: SEQUENCE verb -> BindChannel / SEQUENCE_TRANSITION
            let roll5 = Roll {
                index: 5,
                line_number: 5,
                text: "[ ROLL 5 ] : [ Sequence ] : [ Character` } --> SEQUENCE [ Text ] = [ Text` ]".to_string(),
                status: QuadState::Proceed,
                is_head: false,
            };
            let inst5 = asm.assemble_roll(&roll5, 4, "CharacterTape", &mut syms, &mut off, &mut commit).expect("Assemble
ROLL 5");
            assert_eq!(inst5.op_code, "SEQUENCE_TRANSITION");
            assert_eq!(inst5.op, MachineOp::BindChannel);

            // 5. MMIO addresses: FROM ::@ 0x03F8 and INTO ::@ 0x000B8000
            let roll_uart = Roll {
                index: 6,
                line_number: 6,
                text: "[ COM1 ] : FROM ::@ 0x03F8".to_string(),
                status: QuadState::Proceed,
                is_head: false,
            };
            let inst_uart = asm.assemble_roll(&roll_uart, 5, "CharacterTape", &mut syms, &mut off, &mut
commit).expect("Assemble UART");
            assert_eq!(inst_uart.address, Some(0x03F8));
            assert_eq!(inst_uart.register, Some(MachineRegister::FROM));

            let roll_vga = Roll {
                index: 7,
                line_number: 7,
                text: "[ VGA ] : INTO ::@ 0x000B8000".to_string(),
                status: QuadState::Proceed,
                is_head: false,
            };
            let inst_vga = asm.assemble_roll(&roll_vga, 6, "CharacterTape", &mut syms, &mut off, &mut
commit).expect("Assemble VGA");
            assert_eq!(inst_vga.address, Some(0x000B8000));
            assert_eq!(inst_vga.register, Some(MachineRegister::INTO));
        }
    }
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/mod.rs RS | 94 LINES | 474 WORDS | 4.3 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Physical Assembler Subsystem Identity, Public Facade & Architectural Invariants )
// ============================================================================
//! OmniCode Runtime: Physical Assembler Subsystem Facade
//!
//! # Architectural Scope & Purpose:
//! The Physical Assembler acts as the transduction engine between abstract
//! ontological archetypes (`+`) and concrete silicon memory layouts (`-`).
//! It decomposes into four discrete domain modules:
//! 1. `universal`: Stateless hardware instruction set architecture ([`MachineOp`]),
//!    named physical registers ([`MachineRegister`]), 64-bit balanced-ternary machine word
//!    synthesis ([`synthesize_machine_word`]), and core data containers ([`PhysicalSymbol`], [`AssembledTape`]).
//! 2. `lowering`: Procedural `.tape` and declarative `.omni` statement compilation,
//!    memory partition allocation (`@ 0x0000`), hardware register binding, and delimiter stack validation.
//! 3. `transpiler`: Multi-target code emission (annotated assembly listings, silicon hex words,
//!    IDE hovercard JSON ASTs, bidirectional `.tape` <-> `.omni` conversion, 16-bit Real Mode MBR bootloader,
//!    and 32-bit Multiboot ELF assembly).
//! 4. `core`: Main [`OmniAssembler`] coordinator struct, lifecycle methods, and comprehensive test suite.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of physical machine code onto bounded hardware substrates).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The inviolable preservation of historical silicon addresses, MMIO ports, and opcodes).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Complete Compatibility**: Re-exports all universal ISA definitions, memory symbols,
//!    transpilers, and the coordinator struct to preserve 100% backward compatibility for all callers.
// ============================================================================

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Module Hierarchy Declarations & Public Facade Exports )
// ============================================================================
pub mod universal;
pub mod lowering;
pub mod transpiler;
pub mod core;

// Re-export universal ISA opcodes, registers, and symbols
pub use universal::{
    MachineOp,
    MachineRegister,
    PhysicalSymbol,
    AssembledInstruction,
    AssembledTape,
    DEFAULT_PARTITION_CELL_CAPACITY,
    DEFAULT_ASSEMBLER_DEBUGGER_DEPTH,
    pronoun_to_register,
    pronoun_to_op,
    polarity_to_op,
    compute_word_checksum,
    synthesize_machine_word,
};

// Re-export transpilers and formatters
pub use transpiler::{
    format_assembly_listing,
    format_silicon_words,
    format_tape_json,
    transpile_tape_to_omni,
    transpile_omni_to_tape,
    transpile_tape_to_asm,
    transpile_tape_to_multiboot_asm,
    transpile_omni_to_asm,
    transpile_tape_to_c,
```

```
    transpile_tape_to_lua,
};

// Re-export main coordinator struct
pub use core::OmniAssembler;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Proofs & Invariants )
// ============================================================================
// The Assembler subsystem enforces structural delimiter closure via `OmniDebugger`,
// allocates memory offsets deterministically within storehouse partitions,
// and guarantees that all compiled tapes seal in affirmative `+1:PROCEED`.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification & Module Closure )
// ============================================================================
// Physical Assembler Subsystem facade verified and sealed in affirmative PROCEED.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/lowering/mod.rs

RS | 46 LINES | 276 WORDS | 2.4 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Lowering Subsystem Identity, Invariants & Scripture Anchors )
// ============================================================================
//! OmniCode Runtime: Assembler Lowering Subsystem
//!
//! Provides the AST and tape statement lowering pipelines:
//! - `roll`: Lowers atomic procedural roll statements into physical `AssembledInstruction`s.
//! - `tape`: Lowers entire procedural `.tape` process ribbons into `AssembledTape`.
//! - `omni`: Lowers declarative 4-Block `.omni` spatial architecture reels into `AssembledTape`.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of physical machine code onto bounded hardware substrates).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic roll lowering invariants before constructing system tapes).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The inviolable preservation of historical silicon addresses, MMIO ports, and opcodes).
// ============================================================================

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Module Declarations for Lowering Subsystems )
// ============================================================================
pub mod roll;
pub mod tape;
pub mod omni;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Proofs & Lowering Invariants )
// ============================================================================
// Lowering methods (`assemble_roll`, `assemble_tape`, `assemble_omni`) are attached
// directly to `OmniAssembler` within their respective domain modules.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification & Module Closure )
// ============================================================================
// Lowering module sealed in affirmative PROCEED.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/lowering/omni.rs

RS | 227 LINES | 834 WORDS | 10.2 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Declarative 4-Block Spatial Reel Lowering Engine & Symbol Allocation )
// ============================================================================
//! OmniCode Runtime: Declarative Omni Reel Lowering
//!
//! # Architectural Scope & Purpose:
//! Lowers a declarative 4-Block spatial architecture reel (`.omni`) into physical
//! machine layout:
//! 1. Tokenizes and validates boundary AST physics via [`Lexer`] and [`Parser`].
//! 2. Extracts `Module` or `Manifest` identity and creates a virtual `TapeHead`.
//! 3. Synthesizes physical symbol records (`PhysicalSymbol`) from architectural definitions.
//! 4. Lowers operational lines into machine words ([`Word64`]).
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
// ============================================================================

use std::collections::HashMap;

use crate::core::operators::Polarity;
use crate::core::tape::{Roll, TapeHead};
use crate::core::ternary::QuadState;
use crate::diagnostics::logger::FailpointType;
use crate::parser::lexer::{Lexer, TokenType};
use crate::parser::parser::Parser;
use crate::runtime::assembler::core::OmniAssembler;
use crate::runtime::assembler::universal::types::{
    AssembledInstruction,
    AssembledTape,
    PhysicalSymbol,
};

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( assemble_omni Method Implementation )
// ============================================================================

impl OmniAssembler {
    /// Assembles a declarative 4-Block .omni spatial source reel into physical machine layout.
    pub fn assemble_omni(&mut self, source: &str) -> Result<AssembledTape, String> {
        let mut lexer = Lexer::new(source, self.logger.clone());
        let tokens = match lexer.tokenize() {
            Ok(t) => t,
            Err(e) => {
                self.logger.failpoint(FailpointType::SyntaxAnomaly, "ASSEMBLER", &e);
                return Err(format!("Omni lexical analysis error: {e}"));
            }
        };

        // Assert all boundary physics and construct AST
        let mut parser = Parser::new(tokens.clone(), self.debugger.clone());
        let _prog = match parser.parse_program() {
            Ok(p) => p,
            Err(e) => {
                self.logger.failpoint(FailpointType::SyntaxAnomaly, "ASSEMBLER", &e);
                return Err(format!("Omni program parse error: {e}"));
            }
        };

        // Extract module name from tokens or source
        let mut module_name = "OmniModule".to_string();
        for (i, tok) in tokens.iter().enumerate() {
```

```
            if let TokenType::Identifier(ref id) = tok.token_type {
                if id == "Module" || id == "Manifest" {
                    for lookahead in tokens.iter().skip(i + 1).take(5) {
                        if let TokenType::Identifier(ref name) = lookahead.token_type {
                            module_name = name.clone();
                            break;
                        }
                    }
                }
            }
        }

        // ======================================================================
        // BLOCK 3: BODY BLOCK
        // ( Symbol Table Construction & Instruction Roll Lowering )
        // ======================================================================
        let mut symbols: HashMap<String, PhysicalSymbol> = HashMap::new();
        let mut instructions: Vec<AssembledInstruction> = Vec::new();
        let mut next_available_offset: usize = 0;
        let mut commit_status = QuadState::Await;

        let mut line_index = 0;
        for (line_no, raw_line) in source.lines().enumerate() {
            let line = raw_line.trim();
            if line.is_empty() || line.starts_with('#') {
                continue;
            }
            if line == "Opening {" || line == "Body [" || line == "Closing {" || line == "}" || line == "]" {
                continue;
            }

            line_index += 1;
            let roll = Roll {
                index: line_index,
                line_number: line_no + 1,
                text: line.to_string(),
                status: QuadState::Proceed,
                is_head: false,
            };

            // Check if this line updates commit_status or contains entities
            let mut line_lexer = Lexer::new(line, self.logger.clone());
            if let Ok(line_toks) = line_lexer.tokenize() {
                let is_tripwire_line = line.starts_with("[ Tripwire ]") || line.starts_with("[Tripwire]");
                for (t_idx, tok) in line_toks.iter().enumerate() {
                    if !is_tripwire_line {
                        if let TokenType::Identifier(ref ident) = tok.token_type {
                            let is_commit_status = ident == "commit_status"
                                || (ident == "Commit" && line_toks.get(t_idx + 1).map_or(false, |next| {
                                    if let TokenType::Identifier(ref n) = next.token_type {
                                        n == "Status"
                                    } else {
                                        false
                                    }
                                }));
                            if is_commit_status {
                                let mut resolved = QuadState::Proceed;
                                for lookahead in line_toks.iter().skip(t_idx + 1) {
                                    match &lookahead.token_type {
                                        TokenType::Ternary(qs) => {
                                            resolved = *qs;
                                            break;
                                        }
                                        TokenType::AffixedIdent { name, .. } | TokenType::Identifier(name) => {
                                            let upper = name.to_uppercase();
                                            if upper.starts_with("PROCEED") {
                                                resolved = QuadState::Proceed;
                                                break;
                                            } else if upper.starts_with("ROLLBACK") {
                                                resolved = QuadState::Rollback;
                                                break;
                                            } else if upper.starts_with("AWAIT") {
                                                resolved = QuadState::Await;
                                                break;
                                            } else if upper.starts_with("HALT") {
                                                resolved = QuadState::Halt;
                                                break;
                                            }
```

```
                                        }
                                        _ => {}
                                    }
                                }
                                commit_status = resolved;
                            }
                        }
                    }

                    if let TokenType::AffixedIdent { name, polarity, .. } = &tok.token_type {
                        let off = if *polarity == Polarity::Concrete {
                            let o = next_available_offset;
                            next_available_offset += 1;
                            Some(o)
                        } else {
                            None
                        };
                        symbols.entry(name.clone()).or_insert_with(|| PhysicalSymbol {
                            name: name.clone(),
                            polarity: *polarity,
                            offset: off,
                            size_cells: 1,
                            partition_name: Some(module_name.clone()),
                        });
                    }
                }

                // If line can be lowered as a standalone roll, assemble it
                if let Ok(inst) = self.assemble_roll(
                    &roll,
                    instructions.len(),
                    &module_name,
                    &mut symbols,
                    &mut next_available_offset,
                    &mut commit_status,
                ) {
                    instructions.push(inst);
                }
            }
        }

        let head = TapeHead {
            name: module_name.clone(),
            who: "OmniAssembler".to_string(),
            what: "DeclarativeReel".to_string(),
            when_clause: "Compilation".to_string(),
            where_clause: module_name.clone(),
            why: "SpatialDeclarativeAssembly".to_string(),
            how: commit_status.macro_op().to_string(),
            required_partition_size: 64.max(next_available_offset),
            raw_text: format!("[Reel.{}]", module_name),
        };

        let partition_size = head.required_partition_size;

        self.logger.log(
            "ASSEMBLER",
            format!("Reel [{}] assembled: {} instructions, {} symbols", head.name, instructions.len(), symbols.len()),
        );

        Ok(AssembledTape {
            head,
            partition_size,
            symbols,
            instructions,
            commit_status,
        })
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status & Module Closure )
// ============================================================================
// Omni reel lowering engine verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/lowering/roll.rs

RS | 387 LINES | 1403 WORDS | 17.3 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Atomic Single-Roll Lowering Pipeline & Hardware Allocation Invariants )
// ============================================================================
//! OmniCode Runtime: Atomic Roll Lowering Engine
//!
//! # Architectural Scope & Purpose:
//! Lowers an atomic procedural roll statement into a physical `AssembledInstruction`:
//! 1. Delimiter boundary invariant enforcement via [`OmniDebugger`].
//! 2. Physical memory partition offset allocation (`@ 0x0000`, `next_available_offset`).
//! 3. Hardware CPU register binding (`%head`, `%it`, `%from`, `%into`, etc.).
//! 4. SVO action verb lowering (`INSCRIBE`, `TRANSDUCE`, `PROJECT`, `SEQUENCE`).
//! 5. Covenant commit status resolution (`commit_status [ PROCEED ]`).
//! 6. 64-bit balanced-ternary machine word synthesis ([`Word64`]).
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Silicon Lowering Precision**: Explicit mapping of SVO verbs and MMIO addresses
//!    guarantees consistent machine word generation across all execution targets.
// ============================================================================

use std::collections::HashMap;

use crate::core::boundaries::BoundaryType;
use crate::core::linguistics::PronounKind;
use crate::core::operators::Polarity;
use crate::core::tape::Roll;
use crate::core::ternary::QuadState;
use crate::diagnostics::logger::FailpointType;
use crate::parser::lexer::{Lexer, TokenType};
use crate::runtime::assembler::core::OmniAssembler;
use crate::runtime::assembler::universal::isa::{
    MachineOp,
    MachineRegister,
    pronoun_to_register,
    pronoun_to_op,
    polarity_to_op,
    synthesize_machine_word,
};
use crate::runtime::assembler::universal::types::{
    AssembledInstruction,
    PhysicalSymbol,
};

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( assemble_roll Function Signature & Initializer )
// ============================================================================

impl OmniAssembler {
    /// Lowers an atomic procedural roll into a physical `AssembledInstruction`,
    /// allocating memory partition offsets and hardware register bindings.
    pub fn assemble_roll(
        &mut self,
        roll: &Roll,
        idx: usize,
        where_clause: &str,
        symbols: &mut HashMap<String, PhysicalSymbol>,
        next_available_offset: &mut usize,
        commit_status: &mut QuadState,
    ) -> Result<AssembledInstruction, String> {
```

```
        let mut lexer = Lexer::new(&roll.text, self.logger.clone());
        let tokens = match lexer.tokenize() {
            Ok(t) => t,
            Err(e) => {
                self.logger.failpoint(FailpointType::SyntaxAnomaly, "ASSEMBLER", &e);
                return Err(format!("Roll {idx} assembly error: {e}"));
            }
        };

        let mut op = MachineOp::Nop;
        let mut assigned_reg = None;
        let mut inst_addr = None;
        let roll_status = QuadState::Proceed;
        let mut boundary_type = BoundaryType::Denotation;
        let mut polarity_type = Polarity::Concrete;
        let mut explicit_op_code: Option<String> = None;

        // ========================================================================
        // BLOCK 3: BODY BLOCK
        // ( 1. Invariant Boundary Stack, 2. Token Inspection & Lowering )
        // ========================================================================

        // 1. Enforce structural boundary invariants with OmniDebugger
        self.debugger.boundary_stack.clear();
        for tok in &tokens {
            match tok.token_type {
                TokenType::LBracket => self.debugger.push_boundary('[', tok.line, tok.col),
                TokenType::LBrace => self.debugger.push_boundary('{', tok.line, tok.col),
                TokenType::LAngle => self.debugger.push_boundary('<', tok.line, tok.col),
                TokenType::RBracket => {
                    if let Err(e) = self.debugger.pop_boundary(']', tok.line, tok.col) {
                        return Err(format!("Roll {idx} delimiter error: {e}"));
                    }
                }
                TokenType::RBrace => {
                    if let Err(e) = self.debugger.pop_boundary('}', tok.line, tok.col) {
                        return Err(format!("Roll {idx} delimiter error: {e}"));
                    }
                }
                TokenType::RAngle => {
                    if let Err(e) = self.debugger.pop_boundary('>', tok.line, tok.col) {
                        return Err(format!("Roll {idx} delimiter error: {e}"));
                    }
                }
                _ => {}
            }
        }
        if let Err(e) = self.debugger.assert_all_boundaries_closed() {
            return Err(format!("Roll {idx} boundary leak error: {e}"));
        }

        // 2. Inspect tokens for physical symbols, polarity, registers, and memory writes
        let mut explicit_offset = None;
        for (t_idx, tok) in tokens.iter().enumerate() {
            // Check for @ or ::@ offset locator
            if tok.token_type == TokenType::At || tok.is_hardware_locator() {
                if let Some(next_tok) = tokens.get(t_idx + 1) {
                    if let TokenType::Number(num) = next_tok.token_type {
                        explicit_offset = Some(num as usize);
                    }
                }
            }

            // Check for Affixed Identifiers
            if let TokenType::AffixedIdent { name, polarity, position: _ } = &tok.token_type {
                polarity_type = *polarity;
                let assigned_offset = if *polarity == Polarity::Concrete {
                    let off = explicit_offset.unwrap_or(*next_available_offset);
                    if explicit_offset.is_none() {
                        *next_available_offset += 1;
                    }
                    inst_addr = Some(off);
                    Some(off)
                } else {
                    None
                };

                if let Some(existing) = symbols.get_mut(name) {
```

```
                    if *polarity == Polarity::Concrete {
                        existing.polarity = Polarity::Concrete;
                        if assigned_offset.is_some() {
                            existing.offset = assigned_offset;
                        }
                    }
                } else {
                    symbols.insert(name.clone(), PhysicalSymbol {
                        name: name.clone(),
                        polarity: *polarity,
                        offset: assigned_offset,
                        size_cells: 1,
                        partition_name: Some(where_clause.to_string()),
                    });
                }

                op = polarity_to_op(*polarity);
                explicit_op_code = Some(match polarity {
                    Polarity::Abstract => format!("REF_ABSTRACT_{name}"),
                    Polarity::Concrete => format!("ALLOC_CONCRETE_{name}"),
                });
            }

            // Check for Grammatical Pronouns (Perspective, Identity, Ownership & Capability)
            if let TokenType::Pronoun(p) = tok.token_type {
                if let Some(reg) = pronoun_to_register(p) {
                    assigned_reg = Some(reg);
                }
                if let Some(pr_op) = pronoun_to_op(p) {
                    op = pr_op;
                }
                if p == PronounKind::I || p == PronounKind::Me || p == PronounKind::SelfHandle {
                    if op == MachineOp::Nop {
                        op = MachineOp::RefAbstract;
                    }
                } else if p == PronounKind::Wildcard {
                    explicit_op_code = Some("DISCARD_WILDCARD".to_string());
                }
            }

            // Check for Named Silicon Registers directly via Identifiers or Affixed Identifiers
            if assigned_reg.is_none() {
                match &tok.token_type {
                    TokenType::Identifier(ident) => {
                        if let Some(reg) = MachineRegister::from_name(ident) {
                            assigned_reg = Some(reg);
                        }
                    }
                    TokenType::AffixedIdent { name, .. } => {
                        if let Some(reg) = MachineRegister::from_name(name) {
                            assigned_reg = Some(reg);
                        }
                    }
                    _ => {}
                }
            }

            // Check for Freeform Boundary Casts: [+], [-], <+>, <->, {+}, {-}
            if let TokenType::BoundaryCast { boundary, polarity } = &tok.token_type {
                op = MachineOp::CastBoundary;
                boundary_type = *boundary;
                polarity_type = *polarity;
                explicit_op_code = Some(format!("CAST_{}_{}", boundary.rank_name().replace(' ', "_"),
polarity.symbol()));
            }

            if let TokenType::Identifier(ref ident) = tok.token_type {
                let is_tripwire_roll = tokens.first().map_or(false, |t| {
                    matches!(&t.token_type, TokenType::Identifier(id) if id.eq_ignore_ascii_case("Tripwire"))
                }) || tokens.get(1).map_or(false, |t| {
                    matches!(&t.token_type, TokenType::Identifier(id) if id.eq_ignore_ascii_case("Tripwire"))
                });

                let is_commit_status = !is_tripwire_roll && (ident == "commit_status"
                    || (ident == "Commit" && tokens.get(t_idx + 1).map_or(false, |next| {
                        if let TokenType::Identifier(ref n) = next.token_type {
                            n == "Status"
                        } else {
```

```
                            false
                        }
                    })));
                if is_commit_status {
                    op = MachineOp::SealCommit;
                    explicit_op_code = Some("SEAL_COMMIT".to_string());

                    let mut resolved_state = QuadState::Proceed;
                    for lookahead in tokens.iter().skip(t_idx + 1) {
                        match &lookahead.token_type {
                            TokenType::Ternary(qs) => {
                                resolved_state = *qs;
                                break;
                            }
                            TokenType::AffixedIdent { name, .. } => {
                                let upper = name.to_uppercase();
                                if upper.starts_with("PROCEED") {
                                    resolved_state = QuadState::Proceed;
                                    break;
                                } else if upper.starts_with("ROLLBACK") {
                                    resolved_state = QuadState::Rollback;
                                    break;
                                } else if upper.starts_with("AWAIT") {
                                    resolved_state = QuadState::Await;
                                    break;
                                } else if upper.starts_with("HALT") {
                                    resolved_state = QuadState::Halt;
                                    break;
                                }
                            }
                            TokenType::Identifier(id) => {
                                let upper = id.to_uppercase();
                                if upper.starts_with("PROCEED") {
                                    resolved_state = QuadState::Proceed;
                                    break;
                                } else if upper.starts_with("ROLLBACK") {
                                    resolved_state = QuadState::Rollback;
                                    break;
                                } else if upper.starts_with("AWAIT") {
                                    resolved_state = QuadState::Await;
                                    break;
                                } else if upper.starts_with("HALT") {
                                    resolved_state = QuadState::Halt;
                                    break;
                                }
                            }
                            _ => {}
                        }
                    }
                    *commit_status = resolved_state;
                }
            }

            // Check for Bijective Radix-3 Transduction Equivalence: <=>
            if tok.token_type == TokenType::InvariantEquivalence {
                op = MachineOp::TransduceVector;
                if explicit_op_code.is_none() {
                    explicit_op_code = Some("TRANSDUCE_EQUIVALENCE".to_string());
                }
            }

            // Check for SVO Action Verbs (Character & Object Lifecycle Transitions)
            if let TokenType::Identifier(ref ident) = tok.token_type {
                let upper = ident.to_uppercase();
                match upper.as_str() {
                    "INSCRIBE" => {
                        op = MachineOp::AllocConcrete;
                        if explicit_op_code.is_none() {
                            explicit_op_code = Some("INSCRIBE_CELL".to_string());
                        }
                    }
                    "TRANSDUCE" => {
                        op = MachineOp::TransduceVector;
                        if explicit_op_code.is_none() {
                            explicit_op_code = Some("TRANSDUCE_SUBSTRATE".to_string());
                        }
                    }
                    "PROJECT" => {
```

```
                        op = MachineOp::TransduceVector;
                        if explicit_op_code.is_none() {
                            explicit_op_code = Some("PROJECT_TARGET".to_string());
                        }
                        if assigned_reg.is_none() {
                            assigned_reg = Some(MachineRegister::INTO);
                        }
                    }
                    "SEQUENCE" => {
                        op = MachineOp::BindChannel;
                        if explicit_op_code.is_none() {
                            explicit_op_code = Some("SEQUENCE_TRANSITION".to_string());
                        }
                    }
                    _ => {}
                }
            }

            // Check for directional lowering: -->, v--
            if tok.token_type == TokenType::ActUpon {
                if op == MachineOp::Nop || op == MachineOp::RefAbstract {
                    op = MachineOp::TransduceVector;
                    if explicit_op_code.is_none() {
                        explicit_op_code = Some("TRANSDUCE_VECTOR".to_string());
                    }
                }
            } else if tok.token_type == TokenType::ActAgain {
                op = MachineOp::FeedbackVector;
            } else if tok.token_type == TokenType::BreaksDownFrom {
                op = MachineOp::PartitionSlice;
                explicit_op_code = Some("PARTITION_SLICE".to_string());
            } else if tok.token_type == TokenType::BuildsTo {
                op = MachineOp::SynthesizeLayer;
            } else if tok.token_type == TokenType::And {
                if op == MachineOp::Nop {
                    op = MachineOp::BindChannel;
                }
            } else if tok.token_type == TokenType::But {
                op = MachineOp::AssertInvariant;
            }
        }

        // Fallback for explicit physical MMIO offset if no AffixedIdent set it
        if explicit_offset.is_some() && inst_addr.is_none() {
            inst_addr = explicit_offset;
        }

        // Automatic register binding fallback for hardware MMIO ports
        if assigned_reg.is_none() {
            if let Some(addr) = inst_addr {
                if addr == 0x03F8 {
                    assigned_reg = Some(MachineRegister::FROM);
                } else if addr == 0x000B8000 || addr == 0x01 {
                    assigned_reg = Some(MachineRegister::INTO);
                }
            }
        }

        let op_code = explicit_op_code.unwrap_or_else(|| op.mnemonic().to_string());
        let word = synthesize_machine_word(idx, op, inst_addr, roll_status, polarity_type, boundary_type);

        Ok(AssembledInstruction {
            roll_index: idx + 1,
            source_line: roll.line_number,
            original_text: roll.text.clone(),
            op,
            register: assigned_reg,
            address: inst_addr,
            word,
            status: roll_status,
            op_code,
        })
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status & Module Closure )
```

```
// ============================================================================
// Roll lowering engine verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/lowering/tape.rs RS | 91 LINES | 378 WORDS | 3.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Procedural Process Ribbon Lowering Engine & Memory Partition Sizing )
// ============================================================================
//! OmniCode Runtime: Tape Process Ribbon Lowering
//!
//! # Architectural Scope & Purpose:
//! Lowers an entire temporal procedural process ribbon (`.tape`) into physical
//! silicon machine layout:
//! 1. Ingests the `TapeHead` covenant manifest.
//! 2. Allocates memory partition geometry and symbol tables across all rolls.
//! 3. Sequentially lowers instruction rolls into `Word64` containers.
//! 4. Validates final affirmative commit status (`commit_status`).
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
// ============================================================================

use std::collections::HashMap;

use crate::core::tape::Tape;
use crate::core::ternary::QuadState;
use crate::runtime::assembler::core::OmniAssembler;
use crate::runtime::assembler::universal::types::{
    AssembledInstruction,
    AssembledTape,
    PhysicalSymbol,
};

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( assemble_tape Method Implementation )
// ============================================================================

impl OmniAssembler {
    /// Assembles an entire .tape process ribbon into physical silicon layout.
    pub fn assemble_tape(&mut self, tape: &Tape) -> Result<AssembledTape, String> {
        let head = tape.head.clone();
        let partition_size = head.required_partition_size.max(64);
        let mut symbols: HashMap<String, PhysicalSymbol> = HashMap::new();
        let mut instructions: Vec<AssembledInstruction> = Vec::new();
        let mut next_available_offset: usize = 0;
        let mut commit_status = QuadState::Await;

        // ============================================================================
        // BLOCK 3: BODY BLOCK
        // ( Sequential Roll Iteration & Invariant Aggregation )
        // ============================================================================
        for (idx, roll) in tape.rolls.iter().enumerate() {
            let inst = self.assemble_roll(
                roll,
                idx,
                &head.where_clause,
                &mut symbols,
                &mut next_available_offset,
                &mut commit_status,
            )?;
            instructions.push(inst);
        }

        self.logger.log(
            "ASSEMBLER",
            format!("Tape [{}] assembled: {} instructions, {} symbols", head.name, instructions.len(), symbols.len()),
```

```
        );

        Ok(AssembledTape {
            head,
            partition_size,
            symbols,
            instructions,
            commit_status,
        })
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status & Module Closure )
// ============================================================================
// Tape lowering engine verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/universal/isa.rs RS | 357 LINES | 1693 WORDS | 15.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Physical Machine ISA, Silicon Registers, Word Synthesis & Mappings )
// ============================================================================
//! OmniCode Runtime: Physical Machine ISA & Silicon Registers
//!
//! # Architectural Scope & Purpose:
//! Defines the core Instruction Set Architecture (ISA) of the OmniCode
//! Balanced-Ternary Virtual Machine:
//! 1. `MachineOp`: Hardware machine opcodes mapping abstract grammatical statements
//!    to physical silicon execution.
//! 2. `MachineRegister`: The 13 Named Physical Silicon Registers mapping natural
//!    tape cursors and grammatical perspectives to physical CPU cells.
//! 3. Universal linguistic-to-silicon mappings (`pronoun_to_register`, `pronoun_to_op`,
//!    `polarity_to_op`).
//! 4. 64-bit balanced-ternary machine word synthesis (`synthesize_machine_word`,
//!    `compute_word_checksum`).
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable archetype of physical inscription upon silicon and stone).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic ISA definitions before building lowering engines).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of physical memory boundaries, CPU registers, and opcodes).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Silicon Invariants**: Explicit opcode numbers and register indices guarantee
//!    zero drift between high-level language tapes and bare-metal execution targets.
// ============================================================================

use crate::core::boundaries::BoundaryType;
use crate::core::linguistics::PronounKind;
use crate::core::operators::Polarity;
use crate::core::ternary::QuadState;
use crate::core::word::{Word32Trit, Word64};

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( MachineOp & MachineRegister Enums )
// ============================================================================

/// Hardware Machine Opcodes for the OmniCode Balanced-Ternary Virtual Machine.
#[repr(u8)]
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum MachineOp {
    /// Opcode 0x00: No Operation (Equilibrium state marker).
    Nop             = 0x00,
    /// Opcode 0x01: Reference Abstract Archetype (`+Entity`). Symbolic lookup in Lexicon.
    RefAbstract     = 0x01,
    /// Opcode 0x02: Allocate Concrete Memory Cell (`Entity-`). Reserves physical silicon RAM cell.
    AllocConcrete   = 0x02,
    /// Opcode 0x03: Transduce Vector (`-->`). Forward causal drive, register write, or state propagation.
    TransduceVector = 0x03,
    /// Opcode 0x04: Feedback Vector (`<--`). Reverse causal feedback, sensor tap, or state reflection.
    FeedbackVector  = 0x04,
    /// Opcode 0x05: Partition Slice (`v--`). Carves an isolated, bounded memory partition from a Storehouse.
    PartitionSlice  = 0x05,
    /// Opcode 0x06: Synthesize Layer (`--^`). Ascends abstraction layer or folds micro-states into macro.
    SynthesizeLayer = 0x06,
    /// Opcode 0x07: Bind Channel (`::`). Binds hardware port, MMIO address, or inseparable pipeline channel.
    BindChannel     = 0x07,
    /// Opcode 0x08: Cast Boundary (`[+]`, `[-]`, `<+>`, `<->`). Boundary rank type conversion.
    CastBoundary    = 0x08,
    /// Opcode 0x09: Assert Invariant (`! [ ... ]`). Formal logic proof evaluation; halts on contradiction.
    AssertInvariant = 0x09,
    /// Opcode 0x0A: Introspect Self (`myself`, `ourselves`). Self-audit telemetry and internal state reflection.
    IntrospectSelf  = 0x0A,
```

```
    /// Opcode 0x0B: Recurse Itself (`itself`). Homoiconic tape self-rewrite, loop fixpoint, or quine invocation.
    RecurseItself    = 0x0B,
    /// Opcode 0x0C: Assert Capability (`my`, `your`, `their`). Validates partition ownership and security authority.
    AssertCapability = 0x0C,
    /// Opcode 0x0D: Seal Commit (`commit_status [ PROCEED ]`). Irrevocably seals process covenant state.
    SealCommit       = 0x0D,
    /// Opcode 0x0E: Halt Equilibrium (`0 : HOLD`). Suspends execution head in neutral balanced ternary rest.
    HaltEquilibrium  = 0x0E,
    /// Opcode 0x0F: Trap Veto (`-1 : VETO`). Triggers tripwire collision and initiates affirmative rollback.
    TrapVeto         = 0x0F,
}

impl MachineOp {
    /// Returns the canonical uppercase mnemonic string for assembly listings and telemetry.
    pub fn mnemonic(&self) -> &'static str {
        match self {
            Self::Nop => "NOP",
            Self::RefAbstract => "REF_ABSTRACT",
            Self::AllocConcrete => "ALLOC_CONCRETE",
            Self::TransduceVector => "TRANSDUCE_VECTOR",
            Self::FeedbackVector => "FEEDBACK_VECTOR",
            Self::PartitionSlice => "PARTITION_SLICE",
            Self::SynthesizeLayer => "SYNTHESIZE_LAYER",
            Self::BindChannel => "BIND_CHANNEL",
            Self::CastBoundary => "CAST_BOUNDARY",
            Self::AssertInvariant => "ASSERT_INVARIANT",
            Self::IntrospectSelf => "INTROSPECT_SELF",
            Self::RecurseItself => "RECURSE_ITSELF",
            Self::AssertCapability => "ASSERT_CAPABILITY",
            Self::SealCommit => "SEAL_COMMIT",
            Self::HaltEquilibrium => "HALT_EQUILIBRIUM",
            Self::TrapVeto => "TRAP_VETO",
        }
    }

    /// Resolves a mnemonic string back into its hardware opcode.
    pub fn from_mnemonic(s: &str) -> Option<Self> {
        match s.trim().to_uppercase().as_str() {
            "NOP" => Some(Self::Nop),
            "REF_ABSTRACT" => Some(Self::RefAbstract),
            "ALLOC_CONCRETE" => Some(Self::AllocConcrete),
            "TRANSDUCE_VECTOR" => Some(Self::TransduceVector),
            "FEEDBACK_VECTOR" => Some(Self::FeedbackVector),
            "PARTITION_SLICE" => Some(Self::PartitionSlice),
            "SYNTHESIZE_LAYER" => Some(Self::SynthesizeLayer),
            "BIND_CHANNEL" => Some(Self::BindChannel),
            "CAST_BOUNDARY" => Some(Self::CastBoundary),
            "ASSERT_INVARIANT" => Some(Self::AssertInvariant),
            "INTROSPECT_SELF" => Some(Self::IntrospectSelf),
            "RECURSE_ITSELF" => Some(Self::RecurseItself),
            "ASSERT_CAPABILITY" => Some(Self::AssertCapability),
            "SEAL_COMMIT" => Some(Self::SealCommit),
            "HALT_EQUILIBRIUM" => Some(Self::HaltEquilibrium),
            "TRAP_VETO" => Some(Self::TrapVeto),
            _ => None,
        }
    }

    /// Returns `true` if this opcode represents a forward or reverse directional causality vector.
    pub fn is_causal(&self) -> bool {
        match self {
            Self::TransduceVector | Self::FeedbackVector | Self::PartitionSlice | Self::SynthesizeLayer => true,
            _ => false,
        }
    }

    /// Returns `true` if this opcode reserves or affects physical silicon memory cells.
    pub fn is_memory_allocation(&self) -> bool {
        match self {
            Self::AllocConcrete | Self::PartitionSlice => true,
            _ => false,
        }
    }
}

/// Physical Machine CPU Registers for the Balanced-Ternary Virtual Machine.
#[repr(u8)]
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
```

```rust
pub enum MachineRegister {
    /// Register 0 (%head / %here / %ip): Tape Head / Instruction Pointer / Witness Cursor.
    HEAD = 0,
    /// Register 1 (%home / %origin / %fp): Genesis Base Pointer / Rollback Anchor.
    HOME = 1,
    /// Register 2 (%spool / %owner / %sp): Ribbon Stack Spool / Stack Floor.
    SPOOL = 2,
    /// Register 3 (%it / %acc): Pipeline Antecedent / Primary Accumulator (`it`).
    IT = 3,
    /// Register 4 (%self): 1st-Person Execution Identity (`I`, `me`, `self`).
    SELF = 4,
    /// Register 5 (%caller): 2nd-Person Invoking Principal (`you`, `caller`).
    CALLER = 5,
    /// Register 6 (%other / %swarm): 3rd-Person Peer / Worker Swarm (`other`, `them`).
    OTHER = 6,
    /// Register 7 (%state / %flags): Status & Balanced-Ternary QuadState Register.
    STATE = 7,
    /// Register 8 (%turn / %step): Temporal Cadence & Loop Turn Register (`TURN`).
    TURN = 8,
    /// Register 9 (%bound): Invariant Boundary Watchdog Limit (`BOUND`).
    BOUND = 9,
    /// Register 10 (%from / %input): Inbound Streaming Transducer Port (`FROM`).
    FROM = 10,
    /// Register 11 (%into / %output): Outbound Streaming Transducer Port (`INTO`).
    INTO = 11,
    /// Register 12 (%discard): Anonymous Discard Sink (`DISCARD`, `_`).
    DISCARD = 12,
}

impl MachineRegister {
    /// Canonical and legacy aliases
    pub const HERE: Self = Self::HEAD;
    pub const IP: Self = Self::HEAD;
    pub const ORIGIN: Self = Self::HOME;
    pub const FP: Self = Self::HOME;
    pub const OWNER: Self = Self::SPOOL;
    pub const SP: Self = Self::SPOOL;
    pub const ACC: Self = Self::IT;
    pub const SWARM: Self = Self::OTHER;
    pub const FLAGS: Self = Self::STATE;
    pub const STEP: Self = Self::TURN;
    pub const INPUT: Self = Self::FROM;
    pub const OUTPUT: Self = Self::INTO;

    /// Returns the canonical hardware register identifier (e.g., `%head`, `%home`, `%it`).
    pub fn name(&self) -> &'static str {
        match self {
            Self::HEAD => "%head",
            Self::HOME => "%home",
            Self::SPOOL => "%spool",
            Self::IT => "%it",
            Self::SELF => "%self",
            Self::CALLER => "%caller",
            Self::OTHER => "%other",
            Self::STATE => "%state",
            Self::TURN => "%turn",
            Self::BOUND => "%bound",
            Self::FROM => "%from",
            Self::INTO => "%into",
            Self::DISCARD => "%discard",
        }
    }

    /// Returns the universal ALL CAPS silicon register name (e.g., `HEAD`, `HOME`, `IT`).
    pub fn hardware_name(&self) -> &'static str {
        match self {
            Self::HEAD => "HEAD",
            Self::HOME => "HOME",
            Self::SPOOL => "SPOOL",
            Self::IT => "IT",
            Self::SELF => "SELF",
            Self::CALLER => "CALLER",
            Self::OTHER => "OTHER",
            Self::STATE => "STATE",
            Self::TURN => "TURN",
            Self::BOUND => "BOUND",
            Self::FROM => "FROM",
            Self::INTO => "INTO",
```

```
            Self::DISCARD => "DISCARD",
        }
    }

    /// Returns the physical silicon register numeric index (0 to 12).
    pub fn index(&self) -> u8 {
        *self as u8
    }

    /// Resolves any register string back to its enum variant.
    pub fn from_name(s: &str) -> Option<Self> {
        let clean = s.trim().trim_start_matches(|c| c == '+' || c == '-' || c == '%').to_lowercase();
        match clean.as_str() {
            "head" | "here" | "ip" | "hd" => Some(Self::HEAD),
            "home" | "origin" | "fp" | "hm" => Some(Self::HOME),
            "spool" | "owner" | "sp" => Some(Self::SPOOL),
            "it" | "acc" => Some(Self::IT),
            "self" | "me" | "i" | "sl" => Some(Self::SELF),
            "caller" | "you" | "cl" => Some(Self::CALLER),
            "other" | "swarm" | "them" | "they" | "ot" => Some(Self::OTHER),
            "state" | "flags" | "status" | "st" => Some(Self::STATE),
            "turn" | "step" | "cadence" | "cycle" | "tr" | "tn" => Some(Self::TURN),
            "bound" | "limit" | "watchdog" | "bd" => Some(Self::BOUND),
            "from" | "input" | "in" | "fr" | "source" => Some(Self::FROM),
            "into" | "output" | "out" | "to" | "target" => Some(Self::INTO),
            "discard" | "void" | "_" | "dc" => Some(Self::DISCARD),
            _ => None,
        }
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Universal Linguistic Bridges & Machine Word Synthesis )
// ============================================================================

/// Universal mapping from grammatical pronouns to physical CPU registers.
pub fn pronoun_to_register(p: PronounKind) -> Option<MachineRegister> {
    match p {
        PronounKind::I | PronounKind::Me | PronounKind::SelfHandle => Some(MachineRegister::SELF),
        PronounKind::You | PronounKind::CallerHandle => Some(MachineRegister::CALLER),
        PronounKind::OwnerHandle => Some(MachineRegister::SPOOL),
        PronounKind::It => Some(MachineRegister::IT),
        PronounKind::They | PronounKind::Them | PronounKind::Themselves => Some(MachineRegister::OTHER),
        PronounKind::Wildcard => Some(MachineRegister::DISCARD),
        _ => None,
    }
}

/// Universal mapping from reflexive, recursive, or possessive pronouns to hardware opcodes.
pub fn pronoun_to_op(p: PronounKind) -> Option<MachineOp> {
    match p {
        PronounKind::Myself => Some(MachineOp::IntrospectSelf),
        PronounKind::Itself => Some(MachineOp::RecurseItself),
        PronounKind::Themselves => Some(MachineOp::IntrospectSelf),
        _ if p.is_possessive() => Some(MachineOp::AssertCapability),
        _ => None,
    }
}

/// Universal mapping from formal polarity affixes to memory opcodes.
pub fn polarity_to_op(polarity: Polarity) -> MachineOp {
    match polarity {
        Polarity::Abstract => MachineOp::RefAbstract,
        Polarity::Concrete => MachineOp::AllocConcrete,
    }
}

/// Universal computation of the 7-bit balanced-ternary machine word checksum.
pub fn compute_word_checksum(idx: usize, op: MachineOp) -> u8 {
    ((idx as u8) ^ (op as u8)) & 0x7F
}

/// Universal synthesizer packing payload trits, status, polarity, boundary rank,
/// and checksum into a physical 64-bit balanced-ternary machine word container ([`Word64`]).
pub fn synthesize_machine_word(
    idx: usize,
    op: MachineOp,
```

```
    addr: Option<usize>,
    status: QuadState,
    polarity: Polarity,
    boundary: BoundaryType,
) -> Word64 {
    let payload_trits = addr.map_or_else(Word32Trit::zero, |a| Word32Trit::from_i64(a as i64));
    let checksum = compute_word_checksum(idx, op);
    Word64::new(payload_trits, status, polarity, boundary, checksum)
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests & Verification Invariants )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_machine_op_mnemonics() {
        assert_eq!(MachineOp::Nop.mnemonic(), "NOP");
        assert_eq!(MachineOp::from_mnemonic("ALLOC_CONCRETE"), Some(MachineOp::AllocConcrete));
        assert!(MachineOp::TransduceVector.is_causal());
        assert!(MachineOp::AllocConcrete.is_memory_allocation());
    }

    #[test]
    fn test_machine_register_resolution() {
        assert_eq!(MachineRegister::HEAD.index(), 0);
        assert_eq!(MachineRegister::from_name("%head"), Some(MachineRegister::HEAD));
        assert_eq!(MachineRegister::from_name("ACC"), Some(MachineRegister::IT));
        assert_eq!(MachineRegister::from_name("void"), Some(MachineRegister::DISCARD));
    }

    #[test]
    fn test_machine_word_synthesis() {
        let w = synthesize_machine_word(
            0,
            MachineOp::AllocConcrete,
            Some(0x10),
            QuadState::Proceed,
            Polarity::Concrete,
            BoundaryType::Denotation,
        );
        assert_eq!(w.state(), QuadState::Proceed);
        assert_eq!(w.polarity(), Polarity::Concrete);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/universal/mod.rs RS | 52 LINES | 194 WORDS | 2.0 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal Assembler Subsystem Identity & Public Exports )
// ============================================================================
//! OmniCode Runtime: Universal Assembler ISA & Types Module
//!
//! Re-exports hardware opcodes, machine registers, word synthesis,
//! physical symbols, and assembled tape structures.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**: Tables of testimony, written with the finger of God.
//! - **Proverbs 24:27 (KJV)**: Prepare thy work without, and make it fit for thyself in the field.
//! - **Proverbs 22:28 (KJV)**: Remove not the ancient landmark, which thy fathers have set.
// ============================================================================

pub mod isa;
pub mod types;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Universal ISA & Types )
// ============================================================================
pub use isa::{
    MachineOp,
    MachineRegister,
    pronoun_to_register,
    pronoun_to_op,
    polarity_to_op,
    compute_word_checksum,
    synthesize_machine_word,
};

pub use types::{
    PhysicalSymbol,
    AssembledInstruction,
    AssembledTape,
    DEFAULT_PARTITION_CELL_CAPACITY,
    DEFAULT_ASSEMBLER_DEBUGGER_DEPTH,
};

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Proofs & Invariants )
// ============================================================================
// Universal types are decoupled from lowering pipelines and transpilers.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification & Module Closure )
// ============================================================================
// Universal assembler ISA layer verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/universal/types.rs

RS | 210 LINES | 900 WORDS | 8.2 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Physical Silicon Symbols, Assembled Instructions & Tape Containers )
// ================================================================================
//! OmniCode Runtime: Assembled Machine Artifacts & Symbol Structures
//!
//! # Architectural Scope & Purpose:
//! Defines the core containers produced by the physical silicon lowering engine:
//! 1. `PhysicalSymbol`: Physical memory allocation record for ontological symbols.
//! 2. `AssembledInstruction`: Discrete physical machine instruction container holding
//!    opcodes, silicon offsets, registers, and the 64-bit machine word ([`Word64`]).
//! 3. `AssembledTape`: The complete assembled physical machine tape ready for execution.
//!
//! Also defines baseline partition capacities and debugger depth constants.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Complete Serialization Fidelity**: `AssembledTape` serves as the primary artifact
//!    for both software simulation and bare-metal BIOS/Multiboot execution.
// ================================================================================

use std::collections::HashMap;
use crate::core::operators::Polarity;
use crate::core::tape::TapeHead;
use crate::core::ternary::QuadState;
use crate::core::word::Word64;
use crate::runtime::assembler::universal::isa::{MachineOp, MachineRegister};
use crate::runtime::assembler::transpiler::{
    format_assembly_listing,
    format_silicon_words,
    format_tape_json,
    transpile_tape_to_omni,
    transpile_tape_to_asm,
    transpile_tape_to_multiboot_asm,
    transpile_tape_to_c,
    transpile_tape_to_lua,
};

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Hardware Baseline Constants & PhysicalSymbol Struct )
// ================================================================================

/// Baseline cell capacity reserved for unspecified memory partitions.
pub const DEFAULT_PARTITION_CELL_CAPACITY: usize = 64;
/// Maximum delimiter boundary nesting depth for structural invariant assertions.
pub const DEFAULT_ASSEMBLER_DEBUGGER_DEPTH: usize = 100;

/// Physical allocation record for an ontological symbol in silicon memory.
#[derive(Debug, Clone, PartialEq)]
pub struct PhysicalSymbol {
    /// Ontological name identifier.
    pub name: String,
    /// Formal polarity: Abstract (`+`) or Concrete (`-`).
    pub polarity: Polarity,
    /// Explicit zero-based memory cell offset within the designated partition.
    pub offset: Option<usize>,
    /// Number of contiguous physical RAM cells allocated to this symbol.
    pub size_cells: usize,
    /// Name of the enclosing memory partition or storehouse where allocated.
    pub partition_name: Option<String>,
```

```rust
}

impl PhysicalSymbol {
    /// Returns `true` if this symbol denotes a physical silicon allocation (`-`).
    pub fn is_concrete(&self) -> bool {
        self.polarity == Polarity::Concrete
    }

    /// Returns `true` if this symbol denotes an unallocated abstract archetype (`+`).
    pub fn is_abstract(&self) -> bool {
        self.polarity == Polarity::Abstract
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( AssembledInstruction & AssembledTape Containers )
// ============================================================================

/// A lowered, assembled instruction roll ready for hardware dispatch.
#[derive(Debug, Clone, PartialEq)]
pub struct AssembledInstruction {
    /// 1-based roll position within the execution tape.
    pub roll_index: usize,
    /// 1-based source line number from the original `.tape` or `.omni` document.
    pub source_line: usize,
    /// Raw un-lowered text content of the roll for diagnostics and hovercards.
    pub original_text: String,
    /// Discrete physical machine opcode.
    pub op: MachineOp,
    /// Bound hardware CPU register, if applicable.
    pub register: Option<MachineRegister>,
    /// Target physical memory cell offset, if applicable.
    pub address: Option<usize>,
    /// Packed 64-bit balanced-ternary machine word container.
    pub word: Word64,
    /// QuadState execution status for this instruction (+1:PROCEED).
    pub status: QuadState,
    /// Human-readable operational code string.
    pub op_code: String,
}

/// The complete assembled physical machine tape.
#[derive(Debug, Clone, PartialEq)]
pub struct AssembledTape {
    /// Process covenant manifest header.
    pub head: TapeHead,
    /// Total cell capacity carved for this memory partition.
    pub partition_size: usize,
    /// Resolved physical symbol table mapping identifiers to silicon offsets.
    pub symbols: HashMap<String, PhysicalSymbol>,
    /// Sequential stream of lowered physical machine instructions.
    pub instructions: Vec<AssembledInstruction>,
    /// Final affirmative seal state (+1:PROCEED).
    pub commit_status: QuadState,
}

impl AssembledTape {
    /// Formats the assembled machine tape as a human-readable assembly listing.
    pub fn format_listing(&self) -> String {
        format_assembly_listing(self)
    }

    /// Emits the human-readable physical silicon word stream (Hex, registers, opcodes, QuadState).
    pub fn format_words(&self) -> String {
        format_silicon_words(self)
    }

    /// Emits a structured JSON representation for tooling, IDEs, and diagnostics.
    pub fn format_json(&self) -> String {
        format_tape_json(self)
    }

    /// Transpiles the assembled tape into a declarative 4-Block spatial architecture reel (`.omni`).
    pub fn format_omni(&self) -> String {
        transpile_tape_to_omni(self)
    }

```

```
    /// Transpiles the assembled tape into native bare-metal x86 bootstrap assembly (`.asm`).
    pub fn format_asm(&self) -> String {
        transpile_tape_to_asm(self)
    }

    /// Transpiles the assembled tape into 32-bit Multiboot ELF assembly (`.asm`).
    pub fn format_multiboot_asm(&self) -> String {
        transpile_tape_to_multiboot_asm(self)
    }

    /// Transpiles the assembled tape into a standalone C module (`.c`).
    pub fn format_c(&self) -> String {
        transpile_tape_to_c(self)
    }

    /// Transpiles the assembled tape into a standalone Lua script (`.lua`).
    pub fn format_lua(&self) -> String {
        transpile_tape_to_lua(self)
    }

    /// Emits the raw physical Word64 machine word sequence for hardware dispatch.
    pub fn to_machine_words(&self) -> Vec<Word64> {
        self.instructions.iter().map(|i| i.word).collect()
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests & Verification Proofs )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_physical_symbol_polarity() {
        let sym_abs = PhysicalSymbol {
            name: "Archetype".to_string(),
            polarity: Polarity::Abstract,
            offset: None,
            size_cells: 1,
            partition_name: None,
        };
        assert!(sym_abs.is_abstract());
        assert!(!sym_abs.is_concrete());

        let sym_conc = PhysicalSymbol {
            name: "Concrete".to_string(),
            polarity: Polarity::Concrete,
            offset: Some(0x10),
            size_cells: 1,
            partition_name: Some("Test".to_string()),
        };
        assert!(sym_conc.is_concrete());
        assert!(!sym_conc.is_abstract());
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/transpiler/baremetal.rs

RS | 1113 LINES | 3548 WORDS | 32.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ============================================================================
// Module:           OmniCode.Runtime.Assembler.Transpiler.Baremetal
// System:           MilleniumOS (OmniCode Cognitive Framework)
// Description:      Bare-Metal x86 Dual-State MBR & Diagnostic Engine
// Version:          1.0.0 (Primordial Genesis Inception Baseline)
// Schema:           4-Block Decalogue Transduction Specification v1.0
// Created:          2026-10-11 (Epoch 2026.10 Inscription)
// Updated:          2026-10-16 (RAM Logging, Reg Dumper & Dual-State Audit)
// Authors:          CreativeWorkzStudio LLC / ProfessorSeanEX
// Repository:       https://github.com/Novaking34/OmniCode_Framework
// License:          MIT / Apache-2.0
// Target:           x86 (16-Bit Real Mode BIOS MBR & 32-Bit Multiboot)
// Classification:   Rank 1 Invariant (< Invariant Covenant >)
//
// SCRIPTURE WITNESSES (CANONICAL INVARIANTS):
//   - Exodus 31:18 (KJV):
//     "two tables of testimony, tables of stone, written with the
//      finger of God"
//     (The immutable inscription of atomic runes onto bounded physical tablets)
//   - Proverbs 24:27 (KJV):
//     "Prepare thy work without, and make it fit for thyself in the field;
//      and afterwards build thine house."
//     (The discipline of proving atomic substrates before building systems)
//   - Proverbs 22:28 (KJV):
//     "Remove not the ancient landmark, which thy fathers have set."
//     (The preservation of historical memory boundaries: 0x7C00, 512-byte MBR)
//
// ARCHITECTURAL GUIDANCE FOR DEVELOPERS, COMPILERS & AI MODELS:
//   1. Physical Boundary: Sector 0 is hard-bounded to exactly 512 bytes.
//      Bytes 0-509 hold code/data; bytes 510-511 hold sacred seal 0xAA55.
//   2. Dual-State Duality: State 0 (Safe Mode beacon) guarantees
//      zero-dependency silicon boot; State +1 attempts Stage 2 handoff.
//   3. Transpiler Contract: Emits assembly matching the physical
//      invariants of Track 0 (kernel/bios/boot.asm & multiboot/boot.asm).
//   4. Diagnostic Visibility: Telemetry and audit logging are active during
//      transpilation to verify sector boundaries and architectural covenants.
// ============================================================================

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Environment, Imports, Hardware Constants & Assembly Template Helpers )
// ============================================================================

use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::DiagnosticLogger;
use crate::runtime::assembler::core::OmniAssembler;
use crate::runtime::assembler::universal::types::AssembledTape;

// ----------------------------------------------------------------------------
// HARDWARE INVARIANTS & TOPOLOGICAL CONSTANTS
// ----------------------------------------------------------------------------
/// Physical size of an x86 Master Boot Record sector (Ancient Landmark).
pub const MBR_SECTOR_SIZE: usize = 512;

/// Byte offset where the sacred 0xAA55 boot signature must reside.
pub const MBR_SIGNATURE_OFFSET: usize = 510;

/// Physical memory load address where legacy BIOS places Sector 0.
pub const MBR_LOAD_ADDRESS: u16 = 0x7C00;

/// Base I/O port address for COM1 UART serial controller.
pub const COM1_BASE_PORT: u16 = 0x03F8;

/// Physical memory segment where Stage 2 is loaded (0x1000:0000 = 64 KB mark).
pub const STAGE2_LOAD_SEGMENT: u16 = 0x1000;

/// Memory offset within STAGE2_LOAD_SEGMENT for Stage 2 entry.
pub const STAGE2_LOAD_OFFSET: u16 = 0x0000;

/// Number of disk sectors loaded for Stage 2 kernel payload (2 KB).
pub const STAGE2_SECTOR_COUNT: u8 = 4;
```

```
74
75  /// Multiboot 1 specification magic number required by hypervisors.
76  pub const MULTIBOOT_HEADER_MAGIC: u32 = 0x1BADB002;
77
78  /// Expected loader magic passed in EAX by compliant Multiboot bootloaders.
79  pub const MULTIBOOT_LOADER_MAGIC: u32 = 0x2BADB002;
80
81  /// Standard physical memory load address for 32-bit Multiboot ELF kernels.
82  pub const MULTIBOOT_LOAD_ADDRESS: u32 = 0x00100000;
83
84  /// Physical linear landmark where Stage 2 RAM Log Flight Recorder resides.
85  pub const RAM_LOG_LANDMARK: u32 = 0x00018000;
86
87  /// Physical linear landmark where Multiboot RAM Log Flight Recorder resides.
88  pub const MULTIBOOT_RAM_LOG_LANDMARK: u32 = 0x00108000;
89
90  // ---------------------------------------------------------------------------
91  // TEMPLATE BUILDERS & FORMATTING HELPERS
92  // ( Prevents code duplication and ensures exact adherence to 4-block schema )
93  // ---------------------------------------------------------------------------
94
95  /// Emits the standard Block 1 metadata header for generated assembly files.
96  fn build_asm_metadata_header(process_name: &str, target_desc: &str) -> String {
97      format!(
98  "; ============================================================================
99  ; BLOCK 1: METADATA BLOCK
100 ; ( MilleniumOS {target_desc} )
101 ; ( Transpiled by OmniAssembler from OmniCode Process: {process_name} )
102 ; ============================================================================
103 "
104     )
105 }
106
107 /// Emits the full, verified 16-bit Dual-State MBR bootloader assembly.
108 fn emit_dual_state_mbr_source(process_name: &str) -> String {
109     let mut out = String::with_capacity(4096);
110
111     out.push_str(&build_asm_metadata_header(
112         process_name,
113         "Dual-State Genesis Bootloader - x86 MBR 512-Byte BIOS Sector",
114     ));
115
116     out.push_str(r#"; ARCHITECTURAL DUALITY & THE 512-BYTE ANCHOR POINT:
117 ;   MilleniumOS operates on a balanced ternary dual-state boot architecture:
118 ;   - State 0  (Safe Mode): Hardware truth beacon, ALU proof, 13 silicon regs,
119 ;              zero external dependencies. Runs 100% in Sector 0 (512 bytes).
120 ;   - State +1 (Normal Boot): Uses BIOS INT 0x13 to read Stage 2 from Sector 2+
121 ;              into RAM (0x1000:0000) to awaken the full CPI-SI cognitive model.
122 ;   - State -1 (Fault / Fallback): If disk read fails or a key is held at boot,
123 ;              gracefully retreats to State 0 Safe Mode without crashing.
124 ;
125 ; SCRIPTURAL WITNESSES (COVENANT INVARIANTS):
126 ;   - Exodus 31:18: Immutable inscription onto bounded physical tablets.
127 ;   - Proverbs 24:27: Prepare thy work without, and make it fit in the field.
128 ;   - Proverbs 22:28: Remove not the ancient landmark (0x7C00, 512-byte sector).
129 ;
130 ; TOPOLOGICAL INVARIANTS:
131 ;   1. CPU Mode: 16-Bit Real Mode (segment:offset addressing).
132 ;   2. Telemetry: COM1 UART Serial (0x03F8) at 115200 Baud, 8N1.
133 ;   3. Display: Direct MMIO VGA text buffer at 0x000B8000 & BIOS INT 0x10.
134 ;   4. Truth Verification: Balanced Ternary ALU proof (+1) + (-1) = 0.
135 ;   5. Watchdog: Low-power HLT idle loop preventing CPU lockout/overheating.
136 ; ============================================================================
137
138 ; ============================================================================
139 ; BLOCK 2: ENVIRONMENT / INVARIANTS / SETUP BLOCK
140 ; ( Physical Hardware Origin, Segment Normalization & Stack Architecture )
141 ; ============================================================================
142
143 [BITS 16]
144 [ORG 0x7C00]
145
146 STAGE2_SEG equ 0x1000               ; Stage 2 memory segment (64 KB mark)
147 STAGE2_OFF equ 0x0000               ; Stage 2 memory offset
148 STAGE2_SEC equ 4                    ; Number of sectors to load (2 KB payload)
149
150 start:
151     ; Step 1: Normalize Segments & Initialize the Hardware Stack
```

```
    cli
    xor ax, ax
    mov ds, ax
    mov es, ax
    mov ss, ax
    mov sp, 0x7C00
    mov [boot_drive], dl            ; Preserve physical boot drive from BIOS DL
    mov word [0x0500], 0xAA01       ; RAM Log Landmark: Genesis boot initialized
    mov [0x0502], dl                ; RAM Log Landmark: Logged physical drive
    sti

; ============================================================================
; BLOCK 3: BODY / EXECUTION BLOCK
; ( Hardware Telemetry, Display Inception, ALU Proof & Dual-State Decision )
; ============================================================================

    ; Step 3: Initialize COM1 UART Serial Port (Base Port 0x03F8)
    mov dx, 0x03F9
    xor al, al
    out dx, al

    mov dx, 0x03FB
    mov al, 0x80
    out dx, al

    mov dx, 0x03F8
    mov al, 0x01
    out dx, al

    mov dx, 0x03F9
    xor al, al
    out dx, al

    mov dx, 0x03FB
    mov al, 0x03
    out dx, al

    ; Step 4: Initialize Video Display (80x25 16-Color Text Mode)
    mov ax, 0x0003
    int 0x10

    ; Step 5: Emit Genesis Boot Inception Banner
    mov si, msg_boot
    call print_both

    ; Step 6: Verify Balanced Ternary ALU Equilibrium in Silicon
    mov bx, 1
    mov cx, -1
    add bx, cx
    test bx, bx
    jnz hang_fault

    mov si, msg_alu
    call print_both

    ; Step 7: Dual-State Decision: Check for Safe Mode Keypress Override
    mov ah, 0x01
    int 0x16
    jnz enter_safe_mode

    ; Step 8: Normal Boot Attempt: Load Stage 2 Kernel / CPI-SI Model
    mov ax, STAGE2_SEG
    mov es, ax
    xor bx, bx

    mov ah, 0x02
    mov al, STAGE2_SEC
    xor ch, ch
    mov cl, 2
    xor dh, dh
    mov dl, [boot_drive]
    int 0x13
    jc enter_safe_mode

    cmp word [es:0], 0
    jz enter_safe_mode

    mov si, msg_normal
```

```
    call print_both
    jmp STAGE2_SEG:STAGE2_OFF

; ==========================================================================
; BLOCK 4: SENTINEL / COMPLETION BLOCK
; ( Safe Mode Beacon, Character Transmission & 512-Byte MBR Seal )
; ==========================================================================

enter_safe_mode:
    mov word [0x0500], 0xAA00       ; RAM Log Landmark: State 0 Safe Mode active
    mov si, msg_safe
    call print_both

    ; Arm 13 Silicon Registers to Calibrated Baseline States
    xor ax, ax
    xor bx, bx
    xor cx, cx
    xor dx, dx
    xor si, si
    xor di, di
    xor bp, bp

hang_watchdog:
    hlt
    jmp hang_watchdog

hang_fault:
    mov si, msg_fault
    call print_both
    cli
.fault_loop:
    hlt
    jmp .fault_loop

print_both:
.next_char:
    lodsb
    test al, al
    jz .done

    push ax
    mov ah, 0x0E
    mov bx, 0x000A
    int 0x10
    pop ax

    mov dx, 0x03FD
.wait_tx:
    in al, dx
    test al, 0x20
    jz .wait_tx

    mov al, [si - 1]
    mov dx, 0x03F8
    out dx, al
    jmp .next_char

.done:
    ret

msg_boot:
    db 13, 10
    db "MILLENIUMOS: OMNICODE 32-TRIT OS [x86 MBR]", 13, 10
    db "[BOOT] 16-Bit Genesis: OK", 13, 10, 0

msg_alu:
    db "[ALU ] (+1)+(-1)=0 Eq: OK", 13, 10
    db "[REGS] 13 Regs Armed / Calibrated", 13, 10, 0

msg_normal:
    db "[BOOT] State +1: Stage 2 -> CPI-SI", 13, 10
    db "MILLENIUMOS ONLINE :: PROCEED", 13, 10, 0

msg_safe:
    db "[BOOT] State 0: Safe Mode Beacon", 13, 10
    db "MILLENIUMOS SAFE MODE :: PROCEED", 13, 10, 0

msg_fault:
```

```
    db "[ERR ] ALU Drift: HALT", 13, 10, 0

boot_drive:
    db 0

times 510 - ($ - $$) db 0
dw 0xAA55
"#);

    out
}

/// Emits standard 32-bit Multiboot 1 ELF assembly matching Track 0 kernel.
fn emit_multiboot_elf_source(process_name: &str) -> String {
    let mut out = String::with_capacity(8192);

    out.push_str(&build_asm_metadata_header(
        process_name,
        "Multiboot 1 Direct Hypervisor Kernel & Telemetric Inception",
    ));

    out.push_str(r#"[BITS 32]

section .multiboot
align 4
    dd 0x1BADB002                   ; Multiboot 1 magic
    dd 0x00000003                   ; Flags: align modules (1) + mem info (2)
    dd -(0x1BADB002 + 0x00000003)   ; Checksum: magic + flags + sum == 0

MULTIBOOT_LOADER_MAGIC equ 0x2BADB002 ; Expected EAX from compliant loader
RAM_LOG_OFFSET         equ 0x00108000 ; Flight recorder RAM buffer (32KB above)
CPISI_WORKSPACE_BASE   equ 0x00200000 ; 16-Trit cognitive substrate (2 MB mark)
COM1_PORT              equ 0x03F8     ; Base I/O port for COM1 UART
VGA_TEXT_BUFFER        equ 0x000B8000 ; MMIO video buffer (80x25 text mode)

EVT_MB_ENTRY           equ 0x0001
EVT_MAGIC_VERIFIED     equ 0x0002
EVT_MEM_DISCOVERED     equ 0x0003
EVT_ALU_VERIFIED       equ 0x0004
EVT_REGS_ARMED         equ 0x0005
EVT_CPISI_MOUNTED      equ 0x0006
EVT_CASCADE_BIOS       equ 0x0007
EVT_MB_SAFE_MODE       equ 0xEEEE

section .text
global _start

_start:
    cli

    mov [saved_magic], eax
    mov [saved_info_ptr], ebx
    mov esp, stack_top

    mov dx, 0x03F9
    xor al, al
    out dx, al

    mov dx, 0x03FB
    mov al, 0x80
    out dx, al

    mov dx, 0x03F8
    mov al, 0x01
    out dx, al

    mov dx, 0x03F9
    xor al, al
    out dx, al

    mov dx, 0x03FB
    mov al, 0x03
    out dx, al

    mov esi, msg_banner
    call print_serial32

    mov esi, vga_banner
```

```
    call print_vga32

    cmp dword [saved_magic], MULTIBOOT_LOADER_MAGIC
    jne enter_multiboot_safe_mode

    mov esi, msg_magic_ok
    call print_serial32

    mov ebx, [saved_info_ptr]
    test ebx, ebx
    jz .skip_mem_audit

    test dword [ebx], 0x01
    jz .skip_mem_audit

    mov eax, [ebx + 4]
    mov [mem_lower_kb], eax
    mov eax, [ebx + 8]
    mov [mem_upper_kb], eax

    mov esi, msg_mem_discovered
    call print_serial32

.skip_mem_audit:

    mov ebx, 1
    mov ecx, -1
    add ebx, ecx
    test ebx, ebx
    jnz hang_fault

    mov esi, msg_alu_ok
    call print_serial32

    mov dword [RAM_LOG_OFFSET], 0

    mov eax, EVT_MB_ENTRY
    mov ebx, [saved_magic]
    mov ecx, [saved_info_ptr]
    call log_event32

    mov eax, EVT_MAGIC_VERIFIED
    mov ebx, [saved_magic]
    mov ecx, 0
    call log_event32

    mov eax, EVT_MEM_DISCOVERED
    mov ebx, [mem_lower_kb]
    mov ecx, [mem_upper_kb]
    call log_event32

    mov eax, EVT_ALU_VERIFIED
    mov ebx, 0
    mov ecx, 0
    call log_event32

    mov esi, msg_ram_log_armed
    call print_serial32

    call dump_registers32

    xor eax, eax
    xor ebx, ebx
    xor ecx, ecx
    mov edx, COM1_PORT
    xor esi, esi
    xor edi, edi
    xor ebp, ebp

    mov eax, EVT_REGS_ARMED
    call log_event32

    mov esi, msg_regs_armed
    call print_serial32

    mov eax, EVT_CPISI_MOUNTED
    mov ebx, 16
    mov ecx, 43046721
```

```
    call log_event32

    mov esi, msg_cpisi_mounted
    call print_serial32

    mov esi, msg_online
    call print_serial32

hang_watchdog:
    hlt
    jmp hang_watchdog

enter_multiboot_safe_mode:
    mov eax, EVT_MB_SAFE_MODE
    mov ebx, [saved_magic]
    mov ecx, [saved_info_ptr]
    call log_event32

    mov esi, msg_safe_mode
    call print_serial32
    call dump_registers32

    mov esi, msg_cascade_bios
    call print_serial32

    mov eax, EVT_CASCADE_BIOS
    mov ebx, [saved_magic]
    mov ecx, [saved_info_ptr]
    call log_event32

    jmp cascade_down_to_bios

cascade_down_to_bios:
    cli
    mov esi, trampoline_16
    mov edi, 0x00007000
    mov ecx, trampoline_16_len
    rep movsb
    lidt [real_mode_idtr]
    lgdt [gdt_real_ptr]
    jmp 0x08:0x7000

.cascade_fallback_reset:
    mov al, 0xFE
    out 0x64, al
    hlt
    jmp .cascade_fallback_reset

hang_fault:
    mov esi, msg_fault
    call print_serial32
    cli
.fault_loop:
    hlt
    jmp .fault_loop

log_event32:
    pushad
    mov edx, [RAM_LOG_OFFSET]
    cmp edx, 64
    jae .log_done
    lea edi, [RAM_LOG_OFFSET + 4]
    shl edx, 4
    add edi, edx
    mov [edi + 0], eax
    mov [edi + 4], ebx
    mov [edi + 8], ecx
    mov dword [edi + 12], 0
    inc dword [RAM_LOG_OFFSET]
.log_done:
    popad
    ret

putc_serial32:
    push edx
    push eax
    mov dx, 0x03FD
.wait_tx:
```

```
    in al, dx
    test al, 0x20
    jz .wait_tx
    pop eax
    mov dx, 0x03F8
    out dx, al
    pop edx
    ret

print_serial32:
    pushad
.next:
    lodsb
    test al, al
    jz .done
    call putc_serial32
    jmp .next
.done:
    popad
    ret

print_hex32:
    pushad
    mov ecx, 8
    mov ebx, eax
.hex_loop:
    rol ebx, 4
    mov al, bl
    and al, 0x0F
    cmp al, 9
    jbe .is_digit
    add al, ('A' - 10)
    jmp .emit_nibble
.is_digit:
    add al, '0'
.emit_nibble:
    call putc_serial32
    loop .hex_loop
    popad
    ret

dump_registers32:
    pushad
    mov esi, msg_dbg_prefix
    call print_serial32

    mov esi, msg_dbg_eax
    call print_serial32
    mov eax, [esp + 28]
    call print_hex32

    mov esi, msg_dbg_ebx
    call print_serial32
    mov eax, [esp + 16]
    call print_hex32

    mov esi, msg_dbg_ecx
    call print_serial32
    mov eax, [esp + 24]
    call print_hex32

    mov esi, msg_dbg_edx
    call print_serial32
    mov eax, [esp + 20]
    call print_hex32

    mov esi, msg_dbg_esp
    call print_serial32
    mov eax, [esp + 12]
    call print_hex32

    mov esi, msg_dbg_cr0
    call print_serial32
    mov eax, cr0
    call print_hex32

    mov esi, msg_dbg_crlf
    call print_serial32
```

```
    popad
    ret

print_vga32:
    pushad
    mov edi, VGA_TEXT_BUFFER
    mov ah, 0x0A
.vga_loop:
    lodsb
    test al, al
    jz .vga_done
    mov [edi], ax
    add edi, 2
    jmp .vga_loop
.vga_done:
    popad
    ret

section .rodata

msg_banner:
    db 13, 10
    db "========================================", 13, 10
    db "  MILLENIUMOS 32-TRIT OS [MULTIBOOT 1]  ", 13, 10
    db "  Direct Hypervisor Kernel Handshake   ", 13, 10
    db "========================================", 13, 10, 0

msg_magic_ok:
    db "[BOOT] Hypervisor Magic 0x2BADB002:     VERIFIED", 13, 10, 0

msg_mem_discovered:
    db "[BOOT] Physical Memory Discovery:       OK", 13, 10, 0

msg_alu_ok:
    db "[ALU ] Balanced (+1)+(-1)=0 Eq:         VERIFIED", 13, 10, 0

msg_ram_log_armed:
    db "[BOOT] RAM Log Flight Recorder (108000): ARMED", 13, 10, 0

msg_regs_armed:
    db "[REGS] 13 Named Silicon Registers:      ARMED", 13, 10, 0

msg_cpisi_mounted:
    db "[BOOT] CPI-SI Cognitive Substrate:      MOUNTED", 13, 10, 0

msg_online:
    db "========================================", 13, 10
    db "MILLENIUMOS MULTIBOOT ONLINE :: PROCEED", 13, 10
    db "========================================", 13, 10, 0

msg_safe_mode:
    db "[ERR ] Multiboot Magic Invalid: Retreating to Safe Mode", 13, 10
    db "=== MILLENIUMOS MULTIBOOT SAFE MODE ACTIVE ===", 13, 10, 0

msg_cascade_bios:
    db 13, 10
    db "[CASCADE] De-escalating Ladder: Multiboot -> BIOS (16-Bit)...", 13, 10
    db "[CASCADE] Relinquishing Protected Mode -> Seeking 0x7C00 MBR", 13, 10, 0

msg_fault:
    db "[ERR ] ALU Equilibrium Drift in Hardware: HALTED", 13, 10, 0

msg_dbg_prefix:
    db "[DBG ] REGS: ", 0
msg_dbg_eax:
    db "EAX=0x", 0
msg_dbg_ebx:
    db " EBX=0x", 0
msg_dbg_ecx:
    db " ECX=0x", 0
msg_dbg_edx:
    db " EDX=0x", 0
msg_dbg_esp:
    db 13, 10, "[DBG ] REGS: ESP=0x", 0
msg_dbg_cr0:
    db " CR0=0x", 0
msg_dbg_crlf:
    db 13, 10, 0
```

```
vga_banner:
    db "MILLENIUMOS: OMNICODE 32-TRIT OS [MULTIBOOT 1] ONLINE", 0

trampoline_16:
    db 0xb8, 0x10, 0x00
    db 0x8e, 0xd8
    db 0x8e, 0xc0
    db 0x8e, 0xd0
    db 0xbc, 0x00, 0x7c
    db 0x0f, 0x20, 0xc0
    db 0x24, 0xfe
    db 0x0f, 0x22, 0xc0
    db 0xea, 0x00, 0x7c, 0x00, 0x00
trampoline_16_len equ $ - trampoline_16

section .data
saved_magic:        dd 0
saved_info_ptr:     dd 0
mem_lower_kb:       dd 0
mem_upper_kb:       dd 0

align 16
gdt_real:
    dq 0x0000000000000000
gdt_real_cs16:
    dw 0xFFFF, 0x0000
    db 0x00, 0x9A, 0x00, 0x00
gdt_real_ds16:
    dw 0xFFFF, 0x0000
    db 0x00, 0x92, 0x00, 0x00
gdt_real_ptr:
    dw $ - gdt_real - 1
    dd gdt_real

real_mode_idtr:
    dw 0x03FF
    dd 0x00000000

section .bss
align 16
stack_bottom:
    resb 16384
stack_top:
"#);

    out
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Transpiler Transducers, Diagnostic Logger Integration & Auditing )
// ============================================================================

/// Structural audit inspection result for emitted bare-metal assembly.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct BaremetalSectorInspection {
    /// Target CPU mode (e.g., "16-Bit Real Mode BIOS MBR").
    pub target_arch: &'static str,
    /// Whether explicit CPU bitness mode directive is present.
    pub has_bits_directive: bool,
    /// Whether origin load landmark is present ([ORG 0x7C00] or .multiboot).
    pub has_org_landmark: bool,
    /// Whether ancient landmark boot seal (0xAA55 or 0x1BADB002) is present.
    pub has_boot_signature: bool,
    /// Whether State 0 Safe Mode beacon logic is present.
    pub has_safe_mode_beacon: bool,
    /// Whether State +1 Stage 2 / CPI-SI handoff logic is present.
    pub has_stage2_handoff: bool,
    /// Whether in-memory RAM log landmark tracking is present.
    pub has_ram_log_marker: bool,
    /// Whether bidirectional adaptive boot ladder cascade logic is present.
    pub has_cascade_ladder: bool,
    /// Total lines in emitted assembly source.
    pub total_lines: usize,
}

/// Inspects and audits an emitted bare-metal assembly source string for
```

```rust
/// structural compliance with Sector 0 invariants and covenant boundaries.
pub fn inspect_baremetal_asm(asm_src: &str) -> BaremetalSectorInspection {
    let is_real = asm_src.contains("[BITS 16]");
    let is_multi = asm_src.contains("[BITS 32]");
    BaremetalSectorInspection {
        target_arch: if is_real {
            "16-Bit Real Mode BIOS MBR"
        } else if is_multi {
            "32-Bit Protected Mode Multiboot"
        } else {
            "Unknown Architecture"
        },
        has_bits_directive: is_real || is_multi,
        has_org_landmark: asm_src.contains("[ORG 0x7C00]")
            || asm_src.contains("section .multiboot"),
        has_boot_signature: asm_src.contains("0xAA55")
            || asm_src.contains("0x1BADB002"),
        has_safe_mode_beacon: asm_src.contains("enter_safe_mode:")
            || asm_src.contains("enter_multiboot_safe_mode:"),
        has_stage2_handoff: (asm_src.contains("STAGE2_SEG equ 0x1000")
            && asm_src.contains("int 0x13"))
            || asm_src.contains("CPISI_WORKSPACE_BASE"),
        has_ram_log_marker: asm_src.contains("0x0500")
            || asm_src.contains("RAM_LOG_OFFSET"),
        has_cascade_ladder: asm_src.contains("cascade_down_to_bios")
            || asm_src.contains("EVT_CASCADE_UP")
            || asm_src.contains("EVT_CASCADE_BIOS")
            || (asm_src.contains("STAGE2_SEG equ 0x1000")
                && asm_src.contains("enter_safe_mode:")),
        total_lines: asm_src.lines().count(),
    }
}

/// Transpiles an assembled tape into a bare-metal x86 bootstrap assembly source
/// with explicit diagnostic logging to observe sector budgets and invariants.
pub fn transpile_tape_to_asm_with_logger(
    tape: &AssembledTape,
    logger: &DiagnosticLogger,
) -> String {
    logger.log(
        "BAREMETAL_TRANSPILER",
        format!(
            "Inscribing MilleniumOS Dual-State Genesis MBR for process: {}",
            tape.head.name
        ),
    );

    let asm_src = emit_dual_state_mbr_source(&tape.head.name);
    let audit = inspect_baremetal_asm(&asm_src);

    logger.log(
        "BAREMETAL_TRANSPILER",
        format!(
            "Emitted {} lines. Seal 0xAA55: {}, Safe Mode: {}, RAM Log: {}",
            audit.total_lines,
            audit.has_boot_signature,
            audit.has_safe_mode_beacon,
            audit.has_ram_log_marker
        ),
    );

    logger.record_ai_observation(
        "TRANSPILER_INCEPTION",
        "Dual-State MBR 512-Byte Genesis Assembly Inscribed",
        &format!(
            "Target: {}, Lines: {}, Seal: {}",
            audit.target_arch, audit.total_lines, audit.has_boot_signature
        ),
        ["baremetal", "mbr", "genesis", "dual_state", "track0"].as_slice(),
        Some(&tape.head.name),
    );

    asm_src
}

/// Transpiles an assembled tape into a bare-metal x86 bootstrap assembly source
/// file (`.asm`) suitable for direct assembly by NASM into an MBR 512-byte
/// boot sector for MilleniumOS.
```

```rust
pub fn transpile_tape_to_asm(tape: &AssembledTape) -> String {
    let logger = DiagnosticLogger::new(false);
    transpile_tape_to_asm_with_logger(tape, &logger)
}

/// Transpiles an assembled tape into a 32-bit Multiboot ELF assembly source
/// with diagnostic logging for hypervisor handshake observation.
pub fn transpile_tape_to_multiboot_asm_with_logger(
    tape: &AssembledTape,
    logger: &DiagnosticLogger,
) -> String {
    logger.log(
        "BAREMETAL_TRANSPILER",
        format!(
            "Inscribing Multiboot 1 ELF handshake for process: {}",
            tape.head.name
        ),
    );

    let mb_src = emit_multiboot_elf_source(&tape.head.name);
    let audit = inspect_baremetal_asm(&mb_src);

    logger.log(
        "BAREMETAL_TRANSPILER",
        format!(
            "Emitted Multiboot ELF ({} lines). Magic: {}, Safe: {}, RAM: {}",
            audit.total_lines,
            audit.has_boot_signature,
            audit.has_safe_mode_beacon,
            audit.has_ram_log_marker
        ),
    );

    logger.record_ai_observation(
        "TRANSPILER_INCEPTION",
        "Multiboot 1 Direct Hypervisor Assembly Inscribed",
        &format!(
            "Target: {}, Lines: {}, Seal: {}",
            audit.target_arch, audit.total_lines, audit.has_boot_signature
        ),
        ["baremetal", "multiboot", "genesis", "hypervisor", "track0"]
            .as_slice(),
        Some(&tape.head.name),
    );

    mb_src
}

/// Transpiles an assembled tape into a 32-bit Multiboot ELF assembly source
/// file (`.asm`) suitable for direct assembly by NASM for hypervisor handshake.
pub fn transpile_tape_to_multiboot_asm(tape: &AssembledTape) -> String {
    let logger = DiagnosticLogger::new(false);
    transpile_tape_to_multiboot_asm_with_logger(tape, &logger)
}

/// Standalone helper transpiling a raw declarative .omni string directly
/// into x86 bootstrap assembly.
pub fn transpile_omni_to_asm(source: &str) -> Result<String, String> {
    let logger = DiagnosticLogger::new(false);
    let debugger = OmniDebugger::new(logger.clone(), 100);
    let mut assembler = OmniAssembler::new(logger, debugger);
    assembler.transpile_omni_source_to_asm(source)
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests, Verification Invariants & Diagnostic Assertions )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use std::collections::HashMap;
    use crate::core::tape::TapeHead;
    use crate::core::ternary::QuadState;

    fn create_test_tape(name: &str) -> AssembledTape {
        AssembledTape {
```

```
            head: TapeHead {
                name: name.into(),
                who: "Kernel".into(),
                what: "Boot".into(),
                when_clause: "Boot".into(),
                where_clause: "RAM".into(),
                why: "Proof".into(),
                how: "Proceed".into(),
                required_partition_size: 64,
                raw_text: String::new(),
            },
            partition_size: 64,
            symbols: HashMap::new(),
            instructions: Vec::new(),
            commit_status: QuadState::Proceed,
        }
    }

    #[test]
    fn test_transpile_tape_to_asm_dual_state_invariants() {
        let tape = create_test_tape("GenesisTape");
        let asm_src = transpile_tape_to_asm(&tape);

        // Invariant 1: Physical Architecture and Segments
        assert!(
            asm_src.contains("[BITS 16]"),
            "CRITICAL: 16-Bit Real Mode directive [BITS 16] is missing!"
        );
        assert!(
            asm_src.contains("[ORG 0x7C00]"),
            "CRITICAL: Ancient landmark 0x7C00 load address is missing!"
        );

        // Invariant 2: Physical 512-Byte Ancient Landmark Seal
        assert!(
            asm_src.contains("dw 0xAA55"),
            "CRITICAL: MBR sacred boot signature 0xAA55 is missing!"
        );

        // Invariant 3: Dual-State Architecture (State 0 and State +1)
        assert!(
            asm_src.contains("enter_safe_mode:"),
            "DIAGNOSTIC: Safe Mode fallback label enter_safe_mode missing!"
        );
        assert!(
            asm_src.contains("STAGE2_SEG equ 0x1000"),
            "DIAGNOSTIC: Stage 2 load segment definition missing!"
        );
        assert!(
            asm_src.contains("int 0x13"),
            "DIAGNOSTIC: Stage 2 BIOS disk read interrupt (INT 0x13) missing!"
        );
        assert!(
            asm_src.contains("int 0x16"),
            "DIAGNOSTIC: Safe Mode keystroke check (INT 0x16) missing!"
        );

        // Invariant 4: Brand Identity & Provenance
        assert!(
            asm_src.contains("MILLENIUMOS"),
            "DIAGNOSTIC: Operating system identity MILLENIUMOS missing!"
        );
    }

    #[test]
    fn test_transpile_tape_to_asm_with_diagnostic_logger() {
        let tape = create_test_tape("DiagnosticTape");
        let logger = DiagnosticLogger::new(false);
        let asm_src = transpile_tape_to_asm_with_logger(&tape, &logger);

        // Verify output is complete
        assert!(!asm_src.is_empty());

        // Verify logger captured events and cognitive notes
        let events = logger.events();
        assert!(
            events.iter().any(|e| e.channel == "BAREMETAL_TRANSPILER"),
            "DIAGNOSTIC: Transpiler did not log to expected channel!"
```

```
        );

        let notes = logger.cognitive_notes();
        assert!(
            notes.iter().any(|n| n.category == "TRANSPILER_INCEPTION"),
            "DIAGNOSTIC: AI cognitive observation was not recorded!"
        );
    }

    #[test]
    fn test_inspect_baremetal_asm_auditing() {
        let tape = create_test_tape("AuditTape");
        let asm_src = transpile_tape_to_asm(&tape);
        let audit = inspect_baremetal_asm(&asm_src);

        assert_eq!(audit.target_arch, "16-Bit Real Mode BIOS MBR");
        assert!(audit.has_bits_directive);
        assert!(audit.has_org_landmark);
        assert!(audit.has_boot_signature);
        assert!(audit.has_safe_mode_beacon);
        assert!(audit.has_stage2_handoff);
        assert!(audit.has_ram_log_marker);
        assert!(audit.has_cascade_ladder);
        assert!(audit.total_lines > 100);
    }

    #[test]
    fn test_transpile_tape_to_multiboot_invariants() {
        let tape = create_test_tape("HypervisorTape");
        let mb_src = transpile_tape_to_multiboot_asm(&tape);

        // Invariant 1: 32-Bit Protected Mode
        assert!(
            mb_src.contains("[BITS 32]"),
            "CRITICAL: Multiboot requires 32-bit protected mode [BITS 32]!"
        );

        // Invariant 2: Standard Multiboot 1 Header
        assert!(
            mb_src.contains("section .multiboot"),
            "CRITICAL: .multiboot section marker is missing!"
        );
        assert!(
            mb_src.contains("0x1BADB002"),
            "CRITICAL: Official Multiboot 1 magic 0x1BADB002 is missing!"
        );

        // Invariant 3: Hypervisor Entrypoint & Magic Verification
        assert!(
            mb_src.contains("global _start"),
            "CRITICAL: Multiboot global entrypoint _start is missing!"
        );
        assert!(
            mb_src.contains("0x2BADB002"),
            "CRITICAL: Hypervisor covenant magic 0x2BADB002 check missing!"
        );

        // Invariant 4: In-Memory RAM Log Flight Recorder & Register Dumper
        assert!(
            mb_src.contains("RAM_LOG_OFFSET"),
            "CRITICAL: RAM Log Flight Recorder offset missing!"
        );
        assert!(
            mb_src.contains("dump_registers32"),
            "CRITICAL: Register dumper subroutine missing!"
        );
        assert!(
            mb_src.contains("enter_multiboot_safe_mode"),
            "CRITICAL: Safe mode fallback missing!"
        );

        // Invariant 5: Bidirectional Inception Cascade & 16-Bit Trampoline
        assert!(
            mb_src.contains("cascade_down_to_bios"),
            "CRITICAL: Multiboot to BIOS cascade routine missing!"
        );
        assert!(
            mb_src.contains("trampoline_16"),
```

```
            "CRITICAL: 16-bit real-mode return trampoline missing!"
        );
        assert!(
            mb_src.contains("gdt_real"),
            "CRITICAL: 16-bit real-mode GDT descriptors missing!"
        );
    }

    #[test]
    fn test_inspect_baremetal_multiboot_auditing() {
        let tape = create_test_tape("AuditMultibootTape");
        let mb_src = transpile_tape_to_multiboot_asm(&tape);
        let audit = inspect_baremetal_asm(&mb_src);

        assert_eq!(audit.target_arch, "32-Bit Protected Mode Multiboot");
        assert!(audit.has_bits_directive);
        assert!(audit.has_org_landmark);
        assert!(audit.has_boot_signature);
        assert!(audit.has_safe_mode_beacon);
        assert!(audit.has_stage2_handoff);
        assert!(audit.has_ram_log_marker);
        assert!(audit.has_cascade_ladder);
        assert!(audit.total_lines > 100);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/transpiler/c.rs

RS | 297 LINES | 1211 WORDS | 15.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( C Transpile Target: Lowering Assembled Process Tapes to C Backbone Code )
// ============================================================================
//! OmniCode Runtime: C Transpiler Engine
//!
//! # Architectural Scope & Purpose:
//! Transpiles an [`AssembledTape`] process ribbon into self-contained, canonical C source
//! code conforming to the Track 2 Standard Library C Backbone architecture:
//! 1. Implements strict single-cell bounds and 5-trit balanced ternary types.
//! 2. Synthesizes discrete roll step functions for each procedural tape instruction.
//! 3. Generates a standalone `main()` harness that verifies all rolls in sequence.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of physical data into bounded cell tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types in C before assembling complex systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Cross-Substrate Fidelity**: Transpiling to C allows verification that identical
//!    semantics and quad-state transitions hold across simulated, JIT, and bare-metal environments.
// ============================================================================

use crate::runtime::assembler::universal::types::AssembledTape;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Tape to C Transpiler Function Declaration )
// ============================================================================

/// Transpiles an assembled tape into a complete, standalone C module.
pub fn transpile_tape_to_c(tape: &AssembledTape) -> String {
    let mut out = String::new();

    // Block 1: C Source Metadata
    out.push_str("/* ============================================================================\n");
    out.push_str(" * BLOCK 1: METADATA BLOCK\n");
    out.push_str(&format!(" * ( Transpiled C Module: {} )\n", tape.head.name));
    out.push_str(" * ============================================================================\n");
    out.push_str(" * ARCHITECTURAL TOPOLOGY:\n");
    out.push_str(&format!(" *   Target:         {}\n", tape.head.what));
    out.push_str(&format!(" *   Principal:      {}\n", tape.head.who));
    out.push_str(&format!(" *   When:           {}\n", tape.head.when_clause));
    out.push_str(&format!(" *   Where:          {} (Partition: {} cells)\n", tape.head.where_clause,
tape.partition_size));
    out.push_str(&format!(" *   Why:            ({})\n", tape.head.why));
    out.push_str(&format!(" *   How:            {}\n", tape.head.how));
    out.push_str(" *   Witness:        Exodus 31:18, Proverbs 24:27, Proverbs 22:28 (KJV)\n");
    out.push_str(" * ============================================================================ */\n\n");

    // Block 2: C Includes & Core Types
    out.push_str("#include <stdio.h>\n");
    out.push_str("#include <stdint.h>\n");
    out.push_str("#include <stdbool.h>\n");
    out.push_str("#include <stdlib.h>\n");
    out.push_str("#include <string.h>\n\n");

    out.push_str("/* Balanced Ternary Quad-State Definitions */\n");
    out.push_str("typedef int8_t omni_quad_state_t;\n");
    out.push_str("#define OMNI_STATE_PROCEED   ((omni_quad_state_t) 1)\n");
    out.push_str("#define OMNI_STATE_AWAIT     ((omni_quad_state_t) 0)\n");
    out.push_str("#define OMNI_STATE_ROLLBACK  ((omni_quad_state_t)-1)\n");
    out.push_str("#define OMNI_STATE_HALT      ((omni_quad_state_t)-2)\n\n");

    out.push_str("/* Tape Constants */\n");
```

```
    out.push_str(&format!("#define OMNI_PARTITION_CELLS  {}\n", tape.partition_size));
    out.push_str(&format!("#define OMNI_TOTAL_ROLLS      {}\n\n", tape.instructions.len()));

    out.push_str("/* Dynamic Runtime Context for Assessing Rolls */\n");
    out.push_str("typedef struct {\n");
    out.push_str("    uint8_t glyph;\n");
    out.push_str("    int8_t cell_trits[5];\n");
    out.push_str("    int8_t bit_7;\n");
    out.push_str("    uint8_t collision;\n");
    out.push_str("    bool tripwire_armed;\n");
    out.push_str("    bool target_busy;\n");
    out.push_str("    omni_quad_state_t verify_status;\n");
    out.push_str("    omni_quad_state_t last_status;\n");
    out.push_str("} omni_runtime_context_t;\n\n");
    out.push_str("static omni_runtime_context_t g_tape_ctx = {\n");
    out.push_str("    .glyph = 'A',\n");
    out.push_str("    .cell_trits = {-1, 1, 1, -1, 1},\n");
    out.push_str("    .bit_7 = 0,\n");
    out.push_str("    .collision = 0x00,\n");
    out.push_str("    .tripwire_armed = true,\n");
    out.push_str("    .target_busy = false,\n");
    out.push_str("    .verify_status = OMNI_STATE_PROCEED,\n");
    out.push_str("    .last_status = OMNI_STATE_PROCEED\n");
    out.push_str("};\n\n");

    // Block 3: Roll Functions
    out.push_str("/* ============================================================================\n");
    out.push_str(" * BLOCK 3: BODY BLOCK (ROLL IMPLEMENTATIONS)\n");
    out.push_str(" * ============================================================================ */\n\n");

    for inst in &tape.instructions {
        let safe_text = inst.original_text.replace('"', "\\\"");

        out.push_str(&format!("/**\n * @brief Roll {}: {}\n */\n", inst.roll_index, safe_text));
        out.push_str(&format!("static omni_quad_state_t omni_roll_{}(void) {{\n", inst.roll_index));
        out.push_str(&format!("    printf(\"  ==> [Roll {}] {}\\n\");\n", inst.roll_index, safe_text));
        out.push_str(&generate_c_roll_body(&inst.original_text));
        out.push_str("}\n\n");
    }

    // Block 4: Main Execution Harness
    out.push_str("/* ============================================================================\n");
    out.push_str(" * BLOCK 4: CLOSING BLOCK (HARNESS & VERIFICATION)\n");
    out.push_str(" * ============================================================================ */\n\n");
    out.push_str("int main(void) {\n");
    out.push_str("
printf(\"============================================================================\\n\");\n");
    out.push_str(&format!("    printf(\"   OMNICODE C TRANSPILE RUNNER: [{}]\\n\");\n", tape.head.name));
    out.push_str("
printf(\"============================================================================\\n\");\n");
    out.push_str(&format!("    printf(\"    • Process:   [{}]\\n\");\n", tape.head.name));
    out.push_str(&format!("    printf(\"    • Partition: {} cells\\n\");\n", tape.partition_size));
    out.push_str(&format!("    printf(\"    • Rolls:     {}\\n\");\n", tape.instructions.len()));
    out.push_str("    printf(\"------------------------------------------------------------\\n\");\n\n");

    out.push_str("    omni_quad_state_t state = OMNI_STATE_PROCEED;\n\n");

    for inst in &tape.instructions {
        out.push_str(&format!("    state = omni_roll_{}();\n", inst.roll_index));
        out.push_str("    if (state != OMNI_STATE_PROCEED) {\n");
        out.push_str(&format!("        fprintf(stderr, \"[!] Roll {} halted with state %d\\n\", state);\n",
inst.roll_index));
        out.push_str("        return 1;\n");
        out.push_str("    }\n");
    }

    out.push_str("\n    printf(\"------------------------------------------------------------\\n\");\n");
    out.push_str(&format!(
        "    printf(\"[√] Tape execution complete! All {} rolls verified (State: +1:PROCEED)\\n\");\n",
        tape.instructions.len()
    ));
    out.push_str("    return 0;\n");
    out.push_str("}\n");

    out
}

/// Generates dynamic C roll logic assessing instructions and '|' (OR) branches.
```

```
fn generate_c_roll_body(text: &str) -> String {
    let mut out = String::new();
    if text.contains("Bit_7") || text.contains("Bit 7") {
        out.push_str("    /* Assess guard & conditional branches separated by '|' */\n");
        out.push_str("    if (g_tape_ctx.bit_7 == 0) {\n");
        out.push_str("        g_tape_ctx.last_status = OMNI_STATE_PROCEED;\n");
        out.push_str("        return OMNI_STATE_PROCEED;\n");
        out.push_str("    } else if (g_tape_ctx.bit_7 != 0) {\n");
        out.push_str("        /* Alternative '|' branch: Bit 7 nonzero */\n");
        out.push_str("        g_tape_ctx.last_status = OMNI_STATE_ROLLBACK;\n");
        out.push_str("        return OMNI_STATE_ROLLBACK;\n");
        out.push_str("    }\n");
        out.push_str("    return OMNI_STATE_PROCEED;\n");
    } else if text.contains("Collision") {
        out.push_str("    /* Assess collision sentinel & branches via '|' */\n");
        out.push_str("    if (g_tape_ctx.collision == 0x00) {\n");
        out.push_str("        g_tape_ctx.last_status = OMNI_STATE_PROCEED;\n");
        out.push_str("        return OMNI_STATE_PROCEED;\n");
        out.push_str("    } else if (g_tape_ctx.collision == 0x03) {\n");
        out.push_str("        /* Alternative '|' branch: 11 short-circuit */\n");
        out.push_str("        g_tape_ctx.last_status = OMNI_STATE_ROLLBACK;\n");
        out.push_str("        return OMNI_STATE_ROLLBACK;\n");
        out.push_str("    }\n");
        out.push_str("    return OMNI_STATE_PROCEED;\n");
    } else if text.contains("Commit Status") || text.contains("Verify") {
        out.push_str("    /* Assess Fourfold Commit Status state-machine branches via '|' */\n");
        out.push_str("    if (g_tape_ctx.verify_status == OMNI_STATE_PROCEED\n");
        out.push_str("        && g_tape_ctx.tripwire_armed) {\n");
        out.push_str("        g_tape_ctx.last_status = OMNI_STATE_PROCEED;\n");
        out.push_str("        return OMNI_STATE_PROCEED;\n");
        out.push_str("    } else if (g_tape_ctx.target_busy) {\n");
        out.push_str("        g_tape_ctx.last_status = OMNI_STATE_AWAIT;\n");
        out.push_str("        return OMNI_STATE_AWAIT;\n");
        out.push_str("    } else if (g_tape_ctx.last_status == OMNI_STATE_HALT) {\n");
        out.push_str("        return OMNI_STATE_HALT;\n");
        out.push_str("    } else if (!g_tape_ctx.tripwire_armed\n");
        out.push_str("        || g_tape_ctx.verify_status != OMNI_STATE_PROCEED) {\n");
        out.push_str("        g_tape_ctx.last_status = OMNI_STATE_ROLLBACK;\n");
        out.push_str("        return OMNI_STATE_ROLLBACK;\n");
        out.push_str("    }\n");
        out.push_str("    return OMNI_STATE_PROCEED;\n");
    } else if text.contains('|') {
        out.push_str("    /* Assess dynamic conditional branches separated by '|' */\n");
        out.push_str("    if (g_tape_ctx.tripwire_armed\n");
        out.push_str("        && g_tape_ctx.verify_status == OMNI_STATE_PROCEED) {\n");
        out.push_str("        g_tape_ctx.last_status = OMNI_STATE_PROCEED;\n");
        out.push_str("        return OMNI_STATE_PROCEED;\n");
        out.push_str("    } else {\n");
        out.push_str("        g_tape_ctx.last_status = OMNI_STATE_ROLLBACK;\n");
        out.push_str("        return OMNI_STATE_ROLLBACK;\n");
        out.push_str("    }\n");
    } else if text.contains("INSCRIBE") {
        out.push_str("    /* Dynamic Inscription Assessment: Verify payload bounds */\n");
        out.push_str("    if (g_tape_ctx.glyph == 0 && g_tape_ctx.cell_trits[0] == 0) {\n");
        out.push_str("        g_tape_ctx.last_status = OMNI_STATE_AWAIT;\n");
        out.push_str("        return OMNI_STATE_AWAIT;\n");
        out.push_str("    }\n");
        out.push_str("    g_tape_ctx.last_status = OMNI_STATE_PROCEED;\n");
        out.push_str("    return OMNI_STATE_PROCEED;\n");
    } else if text.contains("TRANSDUCE") {
        out.push_str("    /* Dynamic Dual-Substrate Transduction: Verify parity */\n");
        out.push_str("    if (g_tape_ctx.bit_7 != 0 && g_tape_ctx.collision == 0x03) {\n");
        out.push_str("        g_tape_ctx.last_status = OMNI_STATE_ROLLBACK;\n");
        out.push_str("        return OMNI_STATE_ROLLBACK;\n");
        out.push_str("    }\n");
        out.push_str("    g_tape_ctx.last_status = OMNI_STATE_PROCEED;\n");
        out.push_str("    return OMNI_STATE_PROCEED;\n");
    } else if text.contains("PROJECT") {
        out.push_str("    /* Dynamic Target Buffer Projection: Check destination */\n");
        out.push_str("    if (g_tape_ctx.target_busy) {\n");
        out.push_str("        g_tape_ctx.last_status = OMNI_STATE_AWAIT;\n");
        out.push_str("        return OMNI_STATE_AWAIT;\n");
        out.push_str("    }\n");
        out.push_str("    g_tape_ctx.last_status = OMNI_STATE_PROCEED;\n");
        out.push_str("    return OMNI_STATE_PROCEED;\n");
    } else {
        out.push_str("    g_tape_ctx.last_status = OMNI_STATE_PROCEED;\n");
        out.push_str("    return OMNI_STATE_PROCEED;\n");
```

```
    }
    out
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests for C Transpiler )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use crate::core::tape::TapeHead;
    use crate::core::ternary::QuadState;
    use crate::core::word::Word64;
    use crate::runtime::assembler::universal::isa::MachineOp;
    use crate::runtime::assembler::universal::types::AssembledInstruction;

    #[test]
    fn test_transpile_tape_to_c_output() {
        let head = TapeHead {
            name: "TestCTape".to_string(),
            who: "KernelDaemon".to_string(),
            what: "Verify C Transpiler".to_string(),
            when_clause: "Immediate".to_string(),
            where_clause: "Partition_A".to_string(),
            why: "Parity proof".to_string(),
            how: "+1:PROCEED".to_string(),
            required_partition_size: 512,
            raw_text: String::new(),
        };
        let instructions = vec![
            AssembledInstruction {
                roll_index: 1,
                source_line: 1,
                original_text: "A --> A plus 1".to_string(),
                op: MachineOp::TransduceVector,
                register: None,
                address: None,
                word: Word64::zero(),
                status: QuadState::Proceed,
                op_code: "DRIVE".to_string(),
            },
            AssembledInstruction {
                roll_index: 2,
                source_line: 2,
                original_text: "commit_status [ PROCEED ]".to_string(),
                op: MachineOp::SealCommit,
                register: None,
                address: None,
                word: Word64::zero(),
                status: QuadState::Proceed,
                op_code: "SEAL".to_string(),
            },
        ];
        let tape = AssembledTape {
            head,
            symbols: std::collections::HashMap::new(),
            instructions,
            commit_status: QuadState::Proceed,
            partition_size: 512,
        };

        let c_code = transpile_tape_to_c(&tape);
        assert!(c_code.contains("TestCTape"));
        assert!(c_code.contains("omni_roll_1"));
        assert!(c_code.contains("omni_roll_2"));
        assert!(c_code.contains("int main(void)"));
        assert!(c_code.contains("Tape execution complete!"));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/transpiler/json.rs RS | 109 LINES | 475 WORDS | 5.0 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Structured JSON AST Serializer for IDEs, Diagnostics & Toolchains )
// ================================================================================
//! OmniCode Runtime: Structured JSON Assembled Tape Serializer
//!
//! # Architectural Scope & Purpose:
//! Serializes assembled physical machine tapes into structured JSON for consumption
//! by the language server, IDE hovercards, external analysis tools, and telemetry recorders.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
// ================================================================================

use crate::runtime::assembler::universal::types::AssembledTape;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( JSON Serializer Function Declaration )
// ================================================================================

/// Emits a structured JSON representation of an assembled tape for tooling, IDEs, and diagnostics.
pub fn format_tape_json(tape: &AssembledTape) -> String {
    let mut out = String::new();
    out.push_str("{\n");
    out.push_str(&format!("  \"tape\": \"{}\",\n", tape.head.name));
    out.push_str(&format!("  \"principal\": \"{}\",\n", tape.head.who));
    out.push_str(&format!("  \"target\": \"{}\",\n", tape.head.what));
    out.push_str(&format!("  \"partition_size\": {},\n", tape.partition_size));
    out.push_str(&format!("  \"commit_status\": \"{}\",\n", tape.commit_status.macro_op()));
    out.push_str("  \"symbols\": [\n");
    let sym_list: Vec<_> = tape.symbols.values().collect();
    for (i, s) in sym_list.iter().enumerate() {
        let comma = if i + 1 < sym_list.len() { "," } else { "" };
        let off_str = s.offset.map_or("null".to_string(), |o| o.to_string());
        out.push_str(&format!(
            "    {{ \"name\": \"{}\", \"polarity\": \"{}\", \"size\": {}, \"offset\": {} }}{}\n",
            s.name, s.polarity.symbol(), s.size_cells, off_str, comma
        ));
    }
    out.push_str("  ],\n");
    out.push_str("  \"instructions\": [\n");
    for (i, inst) in tape.instructions.iter().enumerate() {
        let comma = if i + 1 < tape.instructions.len() { "," } else { "" };
        let reg_str = inst.register.map_or("null".to_string(), |r| format!("\"{}\"", r.name()));
        let addr_str = inst.address.map_or("null".to_string(), |a| a.to_string());
        out.push_str(&format!(
            "    {{ \"roll\": {}, \"op\": \"{}\", \"reg\": {}, \"address\": {}, \"word\": \"0x{:016X}\", \"status\": \"{}\" }}{}\n",
            inst.roll_index, inst.op.mnemonic(), reg_str, addr_str, inst.word.raw, inst.status.macro_op(), comma
        ));
    }
    out.push_str("  ]\n");
    out.push_str("}\n");
    out
}

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Invariants & Structural Guarantees )
// ================================================================================
// JSON serializers produce deterministic schemas across all platforms.

```

```rust
// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests & Verification Invariants )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use std::collections::HashMap;
    use crate::core::tape::TapeHead;
    use crate::core::ternary::QuadState;

    #[test]
    fn test_format_tape_json() {
        let tape = AssembledTape {
            head: TapeHead {
                name: "JsonTape".into(),
                who: "Kernel".into(),
                what: "Json".into(),
                when_clause: "Compilation".into(),
                where_clause: "RAM".into(),
                why: "Proof".into(),
                how: "Proceed".into(),
                required_partition_size: 64,
                raw_text: String::new(),
            },
            partition_size: 64,
            symbols: HashMap::new(),
            instructions: Vec::new(),
            commit_status: QuadState::Proceed,
        };
        let json_str = format_tape_json(&tape);
        assert!(json_str.contains("\"tape\": \"JsonTape\""));
        assert!(json_str.contains("\"commit_status\": \"PROCEED\""));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/transpiler/listing.rs

RS | 129 LINES | 537 WORDS | 5.9 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Assembly Listing Formatters & Physical Silicon Word Stream Dumps )
// ================================================================================
//! OmniCode Runtime: Assembly Listing & Silicon Word Serializers
//!
//! # Architectural Scope & Purpose:
//! Formats assembled physical machine tapes into annotated human-readable listings
//! and physical silicon word streams for telemetry, debugging, and terminal audits.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Telemetry Readability**: Explicitly formatted listings allow verification of
//!    memory cell offsets, opcode numbers, and QuadState execution status.
// ================================================================================

use crate::runtime::assembler::universal::types::AssembledTape;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Function Declarations for Listings & Word Dumps )
// ================================================================================

/// Formats an assembled machine tape as a human-readable annotated assembly listing.
pub fn format_assembly_listing(tape: &AssembledTape) -> String {
    let mut out = String::new();
    out.push_str("================================================================\n");
    out.push_str(&format!("ASSEMBLED MACHINE TAPE: [{}]\n", tape.head.name));
    out.push_str(&format!("PRINCIPAL: {} | TARGET: {}\n", tape.head.who, tape.head.what));
    out.push_str(&format!("MEMORY PARTITION: {} cells\n", tape.partition_size));
    out.push_str("----------------------------------------------------------------\n");
    out.push_str("PHYSICAL SYMBOL TABLE:\n");
    for (name, sym) in &tape.symbols {
        let offset_str = sym.offset.map_or("N/A".to_string(), |o| format!("@ 0x{:04X} ({})", o, o));
        out.push_str(&format!(
            "  • {:<16} | Polarity: {:<8} | Size: {:>3} cells | Offset: {}\n",
            name, sym.polarity.symbol(), sym.size_cells, offset_str
        ));
    }
    out.push_str("----------------------------------------------------------------\n");
    out.push_str("ASSEMBLED INSTRUCTION ROLLS:\n");
    for inst in &tape.instructions {
        let addr_str = inst.address.map_or("      ".to_string(), |a| format!("@{:04X}", a));
        let reg_str = inst.register.map_or("     ".to_string(), |r| format!("{:<5}", r.name()));
        out.push_str(&format!(
            "  [{:>2}] {} | {} | {:<20} | Word: 0x{:016X} | Status: {}\n        --> {}\n",
            inst.roll_index, addr_str, reg_str, inst.op_code, inst.word.raw, inst.status.macro_op(), inst.original_text
        ));
    }
    out.push_str("================================================================\n");
    out.push_str(&format!("COMMIT SEAL: {}\n", tape.commit_status.macro_op()));
    out
}

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Silicon Word Stream Serializer )
// ================================================================================

/// Emits the human-readable physical silicon word stream (Hex, registers, opcodes, QuadState).
pub fn format_silicon_words(tape: &AssembledTape) -> String {
```

```
    let mut out = String::new();
    out.push_str(&format!("PHYSICAL SILICON WORD STREAM: [{}]\n", tape.head.name));
    out.push_str("-------------------------------------------------------------\n");
    for (idx, inst) in tape.instructions.iter().enumerate() {
        let reg_str = inst.register.map_or("     ".to_string(), |r| format!("{:<5}", r.name()));
        out.push_str(&format!(
            "[{:>3}] 0x{:016X} | {} | {:<20} | Status: {}\n",
            idx + 1,
            inst.word.raw,
            reg_str,
            inst.op.mnemonic(),
            inst.status.macro_op()
        ));
    }
    out.push_str("-------------------------------------------------------------\n");
    out.push_str(&format!("TOTAL WORDS: {}\n", tape.instructions.len()));
    out
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests & Verification Invariants )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use std::collections::HashMap;
    use crate::core::tape::TapeHead;
    use crate::core::ternary::QuadState;

    #[test]
    fn test_format_listing_empty() {
        let tape = AssembledTape {
            head: TapeHead {
                name: "TestTape".into(),
                who: "Kernel".into(),
                what: "Test".into(),
                when_clause: "Boot".into(),
                where_clause: "RAM".into(),
                why: "Proof".into(),
                how: "Proceed".into(),
                required_partition_size: 64,
                raw_text: String::new(),
            },
            partition_size: 64,
            symbols: HashMap::new(),
            instructions: Vec::new(),
            commit_status: QuadState::Proceed,
        };
        let listing = format_assembly_listing(&tape);
        assert!(listing.contains("TestTape"));
        assert!(listing.contains("COMMIT SEAL: PROCEED"));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/transpiler/lua.rs

RS | 297 LINES | 1189 WORDS | 15.8 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Lua Transpile Target: Lowering Assembled Process Tapes to Lua Tape Scripts )
// ================================================================================
//! OmniCode Runtime: Lua Transpiler Engine
//!
//! # Architectural Scope & Purpose:
//! Transpiles an [`AssembledTape`] process ribbon into self-contained, canonical Lua source
//! code conforming to the Track 2 Standard Library Lua Tape architecture:
//! 1. Implements the dynamic runtime companion driving high-level process flows.
//! 2. Synthesizes declarative tape tables with `metadata`, `constants`, and `rolls`.
//! 3. Generates a self-executing test harness stepping through all rolls.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of physical data into bounded cell tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types in Lua before assembling complex systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Cross-Substrate Fidelity**: Transpiling to Lua allows rapid scripting, interactive
//!    experimentation, and integration with dynamic game engines.
// ================================================================================

use crate::runtime::assembler::universal::types::AssembledTape;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Tape to Lua Transpiler Function Declaration )
// ================================================================================

/// Transpiles an assembled tape into a complete, standalone Lua script.
pub fn transpile_tape_to_lua(tape: &AssembledTape) -> String {
    let mut out = String::new();

    // Block 1: Lua Metadata Header
    out.push_str("-- ================================================================================\n");
    out.push_str("-- BLOCK 1: METADATA BLOCK\n");
    out.push_str(&format!("-- ( Transpiled Lua Tape: {} )\n", tape.head.name));
    out.push_str("-- ================================================================================\n");
    out.push_str("-- ARCHITECTURAL TOPOLOGY:\n");
    out.push_str(&format!("--   Target:         {}\n", tape.head.what));
    out.push_str(&format!("--   Principal:      {}\n", tape.head.who));
    out.push_str(&format!("--   When:           {}\n", tape.head.when_clause));
    out.push_str(&format!("--   Where:          {} (Partition: {} cells)\n", tape.head.where_clause,
tape.partition_size));
    out.push_str(&format!("--   Why:            ({})\n", tape.head.why));
    out.push_str(&format!("--   How:            {}\n", tape.head.how));
    out.push_str("--   Witness:        Exodus 31:18, Proverbs 24:27, Proverbs 22:28 (KJV)\n");
    out.push_str("-- ================================================================================\n\n");

    // Block 2: Constants & Engine Setup
    out.push_str("local STATE_PROCEED  =  1\n");
    out.push_str("local STATE_AWAIT    =  0\n");
    out.push_str("local STATE_ROLLBACK = -1\n");
    out.push_str("local STATE_HALT     = -2\n\n");

    out.push_str("local CONSTANTS = {\n");
    out.push_str(&format!("    PARTITION_CELLS = {},\n", tape.partition_size));
    out.push_str(&format!("    TOTAL_ROLLS     = {},\n", tape.instructions.len()));
    out.push_str("    TRITS_PER_CELL  = 5,\n");
    out.push_str("}\n\n");

    // Block 3: Tape Definition
    out.push_str("-- ================================================================================\n");
    out.push_str("-- BLOCK 3: BODY BLOCK (TAPE SPECIFICATION & ROLLS)\n");
```

```
    out.push_str("-- ================================================================================\n\n");

    out.push_str("local Tape = {\n");
    out.push_str("    metadata = {\n");
    out.push_str(&format!("        name      = \"{}\",\n", tape.head.name));
    out.push_str(&format!("        who       = \"{}\",\n", tape.head.who));
    out.push_str(&format!("        what      = \"{}\",\n", tape.head.what));
    out.push_str(&format!("        when      = \"{}\",\n", tape.head.when_clause));
    out.push_str(&format!("        where     = \"{}\",\n", tape.head.where_clause));
    out.push_str(&format!("        partition = {},\n", tape.partition_size));
    out.push_str(&format!("        why       = \"{}\",\n", tape.head.why));
    out.push_str(&format!("        how       = \"{}\",\n", tape.head.how));
    out.push_str("    },\n");
    out.push_str("    constants = CONSTANTS,\n");
    out.push_str("    rolls = {\n");

    for inst in &tape.instructions {
        let safe_text = inst.original_text.replace('"', "\\\"");
        out.push_str(&format!("        -- Roll {}: {}\n", inst.roll_index, safe_text));
        out.push_str("        {\n");
        out.push_str(&format!("            index  = {},\n", inst.roll_index));
        out.push_str(&format!("            opcode = \"{}\",\n", inst.op_code));
        out.push_str(&format!("            text   = \"{}\",\n", safe_text));
        out.push_str("            action = function(ctx)\n");
        out.push_str(&format!(
            "                print(string.format(\"  ==> [Roll %d] %s\", {}, \"{}\"))\n",
            inst.roll_index, safe_text
        ));
        out.push_str(&generate_lua_roll_body(&inst.original_text));
        out.push_str("            end\n");
        out.push_str("        },\n");
    }

    out.push_str("    }\n");
    out.push_str("}\n\n");

    // Block 4: Self-Executing Runner
    out.push_str("-- ================================================================================\n");
    out.push_str("-- BLOCK 4: CLOSING BLOCK (HARNESS & VERIFICATION)\n");
    out.push_str("-- ================================================================================\n\n");
    out.push_str("function Tape.execute()\n");
    out.push_str("    print(\"================================================================================\")\n");
    out.push_str(&format!("    print(\"   OMNICODE LUA TRANSPILE RUNNER: [{}]\")\n", tape.head.name));
    out.push_str("    print(\"================================================================================\")\n");
    out.push_str(&format!("    print(string.format(\"    • Process:   [%s]\", Tape.metadata.name))\n"));
    out.push_str(&format!("    print(string.format(\"    • Partition: %d cells\", Tape.metadata.partition))\n"));
    out.push_str(&format!("    print(string.format(\"    • Rolls:     %d\", #Tape.rolls))\n"));
    out.push_str("    print(\"----------------------------------------------------------------\")\n\n");

    out.push_str("    local ctx = {\n");
    out.push_str("        glyph = 65,\n");
    out.push_str("        cell_trits = {-1, 1, 1, -1, 1},\n");
    out.push_str("        bit_7 = 0,\n");
    out.push_str("        collision = 0,\n");
    out.push_str("        tripwire_armed = true,\n");
    out.push_str("        target_busy = false,\n");
    out.push_str("        verify_status = STATE_PROCEED,\n");
    out.push_str("        last_status = STATE_PROCEED,\n");
    out.push_str("    }\n\n");
    out.push_str("    for i, roll in ipairs(Tape.rolls) do\n");
    out.push_str("        local status = roll.action(ctx)\n");
    out.push_str("        if status ~= STATE_PROCEED then\n");
    out.push_str("            io.stderr:write(string.format(\"[!] Roll %d halted with state %d\\n\", i, status))\n");
    out.push_str("            os.exit(1)\n");
    out.push_str("        end\n");
    out.push_str("    end\n\n");

    out.push_str("    print(\"----------------------------------------------------------------\")\n");
    out.push_str(&format!(
        "    print(string.format(\"[√] Tape execution complete! All %d rolls verified (State: +1:PROCEED)\", #Tape.rolls))\n"
    ));
    out.push_str("    return 0\n");
    out.push_str("end\n\n");

    out.push_str("-- Run tape directly when invoked as a script\n");
    out.push_str("Tape.execute()\n");

```

```
    out
}

/// Generates dynamic Lua roll logic assessing instructions and '|' (OR) branches.
fn generate_lua_roll_body(text: &str) -> String {
    let mut out = String::new();
    if text.contains("Bit_7") || text.contains("Bit 7") {
        out.push_str("                -- Assess guard & conditional branches separated by '|'\n");
        out.push_str("                if ctx.bit_7 == 0 then\n");
        out.push_str("                    ctx.last_status = STATE_PROCEED\n");
        out.push_str("                    return STATE_PROCEED\n");
        out.push_str("                elseif ctx.bit_7 ~= 0 then\n");
        out.push_str("                    ctx.last_status = STATE_ROLLBACK\n");
        out.push_str("                    return STATE_ROLLBACK\n");
        out.push_str("                end\n");
        out.push_str("                return STATE_PROCEED\n");
    } else if text.contains("Collision") {
        out.push_str("                -- Assess collision sentinel & branches via '|'\n");
        out.push_str("                if ctx.collision == 0 then\n");
        out.push_str("                    ctx.last_status = STATE_PROCEED\n");
        out.push_str("                    return STATE_PROCEED\n");
        out.push_str("                elseif ctx.collision == 3 then\n");
        out.push_str("                    ctx.last_status = STATE_ROLLBACK\n");
        out.push_str("                    return STATE_ROLLBACK\n");
        out.push_str("                end\n");
        out.push_str("                return STATE_PROCEED\n");
    } else if text.contains("Commit Status") || text.contains("Verify") {
        out.push_str("                -- Assess Fourfold Commit Status state-machine branches via '|'\n");
        out.push_str("                if ctx.verify_status == STATE_PROCEED and ctx.tripwire_armed then\n");
        out.push_str("                    ctx.last_status = STATE_PROCEED\n");
        out.push_str("                    return STATE_PROCEED\n");
        out.push_str("                elseif ctx.target_busy then\n");
        out.push_str("                    ctx.last_status = STATE_AWAIT\n");
        out.push_str("                    return STATE_AWAIT\n");
        out.push_str("                elseif ctx.last_status == STATE_HALT then\n");
        out.push_str("                    return STATE_HALT\n");
        out.push_str("                elseif not ctx.tripwire_armed or ctx.verify_status ~= STATE_PROCEED then\n");
        out.push_str("                    ctx.last_status = STATE_ROLLBACK\n");
        out.push_str("                    return STATE_ROLLBACK\n");
        out.push_str("                end\n");
        out.push_str("                return STATE_PROCEED\n");
    } else if text.contains('|') {
        out.push_str("                -- Assess dynamic conditional branches separated by '|'\n");
        out.push_str("                if ctx.tripwire_armed and ctx.verify_status == STATE_PROCEED then\n");
        out.push_str("                    ctx.last_status = STATE_PROCEED\n");
        out.push_str("                    return STATE_PROCEED\n");
        out.push_str("                else\n");
        out.push_str("                    ctx.last_status = STATE_ROLLBACK\n");
        out.push_str("                    return STATE_ROLLBACK\n");
        out.push_str("                end\n");
    } else if text.contains("INSCRIBE") {
        out.push_str("                -- Dynamic Inscription Assessment: Verify payload bounds\n");
        out.push_str("                if ctx.glyph == 0 and (ctx.cell_trits and ctx.cell_trits[1] == 0) then\n");
        out.push_str("                    ctx.last_status = STATE_AWAIT\n");
        out.push_str("                    return STATE_AWAIT\n");
        out.push_str("                end\n");
        out.push_str("                ctx.last_status = STATE_PROCEED\n");
        out.push_str("                return STATE_PROCEED\n");
    } else if text.contains("TRANSDUCE") {
        out.push_str("                -- Dynamic Dual-Substrate Transduction: Verify parity\n");
        out.push_str("                if ctx.bit_7 ~= 0 and ctx.collision == 3 then\n");
        out.push_str("                    ctx.last_status = STATE_ROLLBACK\n");
        out.push_str("                    return STATE_ROLLBACK\n");
        out.push_str("                end\n");
        out.push_str("                ctx.last_status = STATE_PROCEED\n");
        out.push_str("                return STATE_PROCEED\n");
    } else if text.contains("PROJECT") {
        out.push_str("                -- Dynamic Target Buffer Projection: Check destination\n");
        out.push_str("                if ctx.target_busy then\n");
        out.push_str("                    ctx.last_status = STATE_AWAIT\n");
        out.push_str("                    return STATE_AWAIT\n");
        out.push_str("                end\n");
        out.push_str("                ctx.last_status = STATE_PROCEED\n");
        out.push_str("                return STATE_PROCEED\n");
    } else {
        out.push_str("                ctx.last_status = STATE_PROCEED\n");
        out.push_str("                return STATE_PROCEED\n");
    }
```

```
    out
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests for Lua Transpiler )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use crate::core::tape::TapeHead;
    use crate::core::ternary::QuadState;
    use crate::core::word::Word64;
    use crate::runtime::assembler::universal::isa::MachineOp;
    use crate::runtime::assembler::universal::types::AssembledInstruction;

    #[test]
    fn test_transpile_tape_to_lua_output() {
        let head = TapeHead {
            name: "TestLuaTape".to_string(),
            who: "KernelDaemon".to_string(),
            what: "Verify Lua Transpiler".to_string(),
            when_clause: "Immediate".to_string(),
            where_clause: "Partition_L".to_string(),
            why: "Parity proof".to_string(),
            how: "+1:PROCEED".to_string(),
            required_partition_size: 256,
            raw_text: String::new(),
        };
        let instructions = vec![
            AssembledInstruction {
                roll_index: 1,
                source_line: 1,
                original_text: "A --> A plus 1".to_string(),
                op: MachineOp::TransduceVector,
                register: None,
                address: None,
                word: Word64::zero(),
                status: QuadState::Proceed,
                op_code: "DRIVE".to_string(),
            },
            AssembledInstruction {
                roll_index: 2,
                source_line: 2,
                original_text: "commit_status [ PROCEED ]".to_string(),
                op: MachineOp::SealCommit,
                register: None,
                address: None,
                word: Word64::zero(),
                status: QuadState::Proceed,
                op_code: "SEAL".to_string(),
            },
        ];
        let tape = AssembledTape {
            head,
            symbols: std::collections::HashMap::new(),
            instructions,
            commit_status: QuadState::Proceed,
            partition_size: 256,
        };

        let lua_code = transpile_tape_to_lua(&tape);
        assert!(lua_code.contains("TestLuaTape"));
        assert!(lua_code.contains("PARTITION_CELLS = 256"));
        assert!(lua_code.contains("function Tape.execute()"));
        assert!(lua_code.contains("Tape.execute()"));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/transpiler/mod.rs

RS | 59 LINES | 217 WORDS | 2.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Multi-Target Transpiler Subsystem Identity & Public Exports )
// ============================================================================
//! OmniCode Runtime: Multi-Target Transpiler Module
//!
//! Provides text assembly listings, physical silicon hex dumps, structured JSON ASTs,
//! bidirectional `.tape` <--> `.omni` translation, and bare-metal x86/Multiboot assembly.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**: Tables of testimony, written with the finger of God.
//! - **Proverbs 24:27 (KJV)**: Prepare thy work without, and make it fit for thyself in the field.
//! - **Proverbs 22:28 (KJV)**: Remove not the ancient landmark, which thy fathers have set.
// ============================================================================

pub mod listing;
pub mod json;
pub mod omni;
pub mod baremetal;
pub mod c;
pub mod lua;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Transpilers & Serializers )
// ============================================================================
pub use listing::{
    format_assembly_listing,
    format_silicon_words,
};

pub use json::format_tape_json;

pub use omni::{
    transpile_tape_to_omni,
    transpile_omni_to_tape,
};

pub use baremetal::{
    transpile_tape_to_asm,
    transpile_tape_to_multiboot_asm,
    transpile_omni_to_asm,
};

pub use c::transpile_tape_to_c;
pub use lua::transpile_tape_to_lua;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Proofs & Invariants )
// ============================================================================
// Transpilers project assembled artifacts to external formats without state mutation.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification & Module Closure )
// ============================================================================
// Multi-target transpiler layer verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/assembler/transpiler/omni.rs RS | 182 LINES | 778 WORDS | 9.2 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Bidirectional Transpilation Between Procedural Tapes & Spatial Omni Reels )
// ================================================================================
//! OmniCode Runtime: Bidirectional Omni Transpiler
//!
//! # Architectural Scope & Purpose:
//! Performs lossless bidirectional translation between the two primary forms
//! of OmniCode source documents:
//! 1. Temporal Procedural Process Ribbons (`.tape`)
//! 2. Declarative 4-Block Spatial Architecture Reels (`.omni`)
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Isomorphic Triad**: Tapes and Reels are mathematical duals. A developer or agent
//!    can author declarative reels and transpile to procedural tapes, or vice-versa.
// ================================================================================

use crate::runtime::assembler::universal::types::AssembledTape;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Tape to Omni Transpiler Function Declaration )
// ================================================================================

/// Transpiles an assembled tape from temporal process ribbon (`.tape`) into
/// a declarative, 4-Block spatial architecture reel (`.omni`).
pub fn transpile_tape_to_omni(tape: &AssembledTape) -> String {
    let mut out = String::new();
    out.push_str("# ================================================================================\n");
    out.push_str("# BLOCK 1: METADATA BLOCK\n");
    out.push_str(&format!("# ( Transpiled Spatial Reel: {} )\n", tape.head.name));
    out.push_str("# ================================================================================\n");
    out.push_str(&format!("Module < {} > ( Transpiled by OmniAssembler ) {{\n", tape.head.name));
    out.push_str("    version: < \"1.0.0\" >,\n");
    out.push_str("    system: < OmniCode.Transpiled >,\n");
    out.push_str(&format!("    principal: < \"{}\" >,\n", tape.head.who));
    out.push_str(&format!("    target: < \"{}\" >,\n", tape.head.what));
    out.push_str(&format!("    partition_capacity: < {} >\n", tape.partition_size));
    out.push_str("}\n\n");

    if !tape.symbols.is_empty() {
        out.push_str("# Physical Silicon Symbol Allocations\n");
        for (name, sym) in &tape.symbols {
            let off_desc = sym.offset.map_or("dynamic".to_string(), |o| format!("@ 0x{:04X}", o));
            let pol_char = sym.polarity.symbol();
            out.push_str(&format!(
                "Entity_{} < PhysicalSymbol > ( Silicon memory cell [{} {}] ) {{\n    polarity: < \"{}\" >,\n
capacity_cells: < {} >\n}}\n\n",
                name, pol_char, off_desc, pol_char, sym.size_cells
            ));
        }
    }

    out.push_str("# ================================================================================\n");
    out.push_str("# BLOCK 2: OPENING BLOCK\n");
    out.push_str("# ================================================================================\n");
    out.push_str("Opening {\n");
    out.push_str("    requires: [\n");
    out.push_str(&format!("        PartitionCapacity : {} cells ::\n", tape.partition_size));
    out.push_str(&format!("        TapeHead : {} [ {} ]\n", tape.head.name, tape.head.who));
```

```
    out.push_str("    ],\n");
    out.push_str("    preconditions: [ +1:PROCEED ]\n");
    out.push_str("}\n\n");

    out.push_str("# ================================================================================\n");
    out.push_str("# BLOCK 3: BODY BLOCK\n");
    out.push_str("# ================================================================================\n");
    out.push_str("Body [\n");
    for inst in &tape.instructions {
        let reg_str = inst.register.map_or(String::new(), |r| format!(" ( Reg: {} )", r.name()));
        out.push_str(&format!("    Roll_{} [ {} ]{}\n", inst.roll_index, inst.original_text, reg_str));
    }
    out.push_str("]\n\n");

    out.push_str("# ================================================================================\n");
    out.push_str("# BLOCK 4: CLOSING BLOCK\n");
    out.push_str("# ================================================================================\n");
    out.push_str("Closing {\n");
    out.push_str("    verify: [\n");
    out.push_str(&format!("        InstructionCount = {} ::\n", tape.instructions.len()));
    out.push_str(&format!("        SymbolCount = {}\n", tape.symbols.len()));
    out.push_str("    ],\n");
    out.push_str(&format!("    commit_status [ {} ]\n", tape.commit_status.macro_op()));
    out.push_str("}\n");

    out
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Omni to Tape Transpiler Function )
// ============================================================================

/// Transpiles an assembled tape from declarative 4-Block spatial architecture reel (`.omni`)
/// into an executable procedural process ribbon (`.tape`).
pub fn transpile_omni_to_tape(tape: &AssembledTape) -> String {
    let mut out = String::new();
    out.push_str("# ================================================================================\n");
    out.push_str("# BLOCK 1: METADATA BLOCK\n");
    out.push_str(&format!("# ( Procedural Process Ribbon Transpiled from Omni Reel: {} )\n", tape.head.name));
    out.push_str("# ================================================================================\n\n");
    out.push_str(&format!(
        "[Tape.{}] : WHO {{{}}} WHAT {{{}}} --> BOOT {{{}}}\n    WHEN {{Scheduled}}\n    WHERE {{{}}} = [ {} ]\n    WHY
( Executable procedural ribbon lowered from declarative 4-block architecture )\n    HOW [ +1 : {} ]\n    :: < ancient_landmark
>\n\n",
        tape.head.name, tape.head.who, tape.head.what, tape.head.name, tape.head.where_clause, tape.partition_size,
tape.commit_status.macro_op()
    ));

    out.push_str("# ================================================================================\n");
    out.push_str("# BLOCK 2: OPENING BLOCK\n");
    out.push_str("# ( Preconditions & Memory Allocations )\n");
    out.push_str("# ================================================================================\n\n");

    out.push_str("# ================================================================================\n");
    out.push_str("# BLOCK 3: BODY BLOCK\n");
    out.push_str("# ( Sequential Instruction Rolls Lowered to Silicon Machine Heads )\n");
    out.push_str("# ================================================================================\n\n");

    for inst in &tape.instructions {
        out.push_str(&format!("roll_{} [ {} ]\n", inst.roll_index, inst.original_text));
    }

    out.push_str("\n# ================================================================================\n");
    out.push_str("# BLOCK 4: CLOSING BLOCK\n");
    out.push_str("# ( Covenant Commit Seal )\n");
    out.push_str("# ================================================================================\n\n");
    out.push_str(&format!("seal_roll [ commit_status [ {} ] ]\n", tape.commit_status.macro_op()));

    out
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests & Verification Invariants )
// ============================================================================

#[cfg(test)]
```

```rust
mod tests {
    use super::*;
    use std::collections::HashMap;
    use crate::core::tape::TapeHead;
    use crate::core::ternary::QuadState;

    #[test]
    fn test_bidirectional_omni_tape_transpilation() {
        let tape = AssembledTape {
            head: TapeHead {
                name: "DualTape".into(),
                who: "Kernel".into(),
                what: "Dual".into(),
                when_clause: "Compilation".into(),
                where_clause: "RAM".into(),
                why: "Proof".into(),
                how: "Proceed".into(),
                required_partition_size: 64,
                raw_text: String::new(),
            },
            partition_size: 64,
            symbols: HashMap::new(),
            instructions: Vec::new(),
            commit_status: QuadState::Proceed,
        };
        let omni_src = transpile_tape_to_omni(&tape);
        assert!(omni_src.contains("BLOCK 1: METADATA BLOCK"));
        assert!(omni_src.contains("Module < DualTape >"));

        let tape_src = transpile_omni_to_tape(&tape);
        assert!(tape_src.contains("[Tape.DualTape]"));
        assert!(tape_src.contains("commit_status [ PROCEED ]"));
    }
}
```